



UiT The Arctic University of Norway

Faculty of Science and Technology
Department of Physics and Technology

Sparse Neural Network Interpretability: A Comparative Analysis of Autoencoders and Transformers

Sebastian Tendstrand Iversen

FYS-3941 Master's Thesis in Applied Physics and Mathematics 30 SP- June.2025



Abstract

As neural networks grow increasingly complex and powerful, understanding their internal representations becomes critical for ensuring safe and reliable AI systems. This thesis addresses a fundamental challenge in mechanistic interpretability: how architectural choices in sparse representation learning shape our ability to understand neural network internals. We present the first comprehensive theoretical and empirical comparison of Sparse Autoencoders (SAEs) and Sparse Transformers (STs), two competing approaches for decomposing neural network activations into interpretable features.

We investigate a unified geometric framework revealing that these architectures, despite solving the same sparse coding problem, operate in fundamentally different spaces. SAEs use ReLU activations to create unbounded sparse features in positive orthants, while STs employ softmax attention to confine features to the probability simplex. These constraints lead to complementary capabilities: SAEs excel at Euclidean separation through magnitude differences, while STs optimize for angular discrimination through competitive dynamics.

Our empirical validation across visual datasets (MNIST and Fashion-MNIST) confirms dramatic performance differences predicted by theory. SAEs achieve $100\text{--}1000\times$ superior Euclidean separation between class representations, while STs demonstrate $10\times$ better angular discrimination. Feature-level analysis reveals that SAEs learn shared, polysemantic features that capture cross-class patterns, whereas STs develop highly specialized, monosemantic features through winner-take-all competition. When applied to GPT-Neo 1.3B language model representations, these performance gaps narrow substantially (to $3\text{--}30\times$), suggesting that architectural biases, while persistent, are moderated by the complexity of real-world applications.

This work establishes that the choice of interpretability architecture is not merely an implementation detail but fundamentally determines what aspects of neural computation we can observe and understand. SAEs provide superior tools for identifying distinct concepts requiring clear on/off behavior, while STs excel at revealing relational structures and relative feature importance. These complementary strengths suggest that future interpretability research should focus not on choosing between architectures but on understanding how to leverage their respective advantages for comprehensive mechanistic understanding.

Acknowledgements

I would like to express my gratitude to my supervisor, Professor Benjamin Ricaud, whose guidance, expertise, and encouragement have been invaluable throughout this research. Benjamin's ability to motivate and inspire, particularly during challenging moments, has been instrumental in bringing this work to completion.

Contents

Abstract	i
Acknowledgements	ii
List of Figures	vii
List of Tables	xi
1 Introduction	1
1.1 Motivation	1
1.2 Sparse Representations	2
1.3 Research Question	2
1.4 Thesis Contributions	2
1.5 Thesis Structure	3
I Background	5
2 Theoretical Foundations	7
2.1 Introduction	7
2.2 Neural Network Interpretability	8
2.2.1 The Black Box Problem	8
2.2.2 Approaches to Interpretability	9
2.3 Sparse Dictionary Learning and Polysemanticity	10
2.3.1 Mathematical Foundations of Sparse Coding	10
2.3.2 Polysemanticity in Neural Networks	11
2.3.3 Sparse Representations as a Solution	12
2.4 Large Language Models and the Need for Interpretability . .	12
2.4.1 Transformer Architecture Fundamentals	12
2.4.2 The Need for Interpretability	12
2.5 Sparse Autoencoders: Theory and Architecture	13
2.5.1 Architectural Formulation	13
2.5.2 Optimization Objective	13
2.5.3 Geometric Properties and Sparsity Mechanism	14
2.5.4 Theoretical Advantages and Limitations	14
2.6 Sparse Transformers: Theoretical Foundations	15

2.6.1	Motivation for Attention-Based Sparsity	15
2.6.2	Theoretical Architecture	16
2.6.3	Sparsity Mechanism in Sparse Transformers	17
2.6.4	Theoretical Properties	18
2.7	Comparative Theoretical Analysis	18
2.7.1	Unified Framework	18
2.7.2	Geometric Comparison	19
2.7.3	Sparsity Mechanism Comparison	20
2.7.4	Optimization Landscape Analysis	20
2.7.5	Interpretability Trade-offs	21
2.7.6	Theoretical Predictions	21
II	Methodology	23
3	Methodology	25
3.1	Introduction	25
3.2	Experimental Framework	26
3.2.1	Dataset Selection	26
3.2.2	Evaluation Metrics	26
3.3	Model Implementations	27
3.3.1	Sparse Autoencoder Configuration	27
3.3.2	Sparse Transformer Implementation	28
3.4	Hyperparameter Grid	30
3.4.1	Systematic Exploration	30
3.4.2	Adaptive Training Schedule	30
3.4.3	Fixed Parameters	30
3.5	Feature Analysis Methodology	30
3.5.1	Feature Importance Quantification	30
3.5.2	Feature Manipulation Protocol	31
3.5.3	Feature Interaction Analysis	31
III	Experiments	33
4	Results	35
4.1	Introduction	35
4.2	Comparative Hyperparameter Analysis	36
4.3	Feature-Level Interpretability Analysis	46
4.3.1	Feature Variation Effects on MNIST	46
4.3.2	Feature Variation Effects on Fashion-MNIST	57
4.3.3	Dataset Complexity and Feature Learning	65
4.4	Application to Large Language Model Representations	66
4.4.1	Experimental Setup	67

4.4.2	Centroid Distance Results	67
4.4.3	Layer-Specific Patterns	69
4.4.4	Comparison with Toy Dataset Results	70
IV	Discussion and conclusion	71
5	Discussion	73
5.1	Introduction	73
5.2	Synthesis of Key Findings	74
5.2.1	Validation of Theoretical Predictions	74
5.2.2	Unexpected Discoveries	74
5.3	Implications for Mechanistic Interpretability	75
5.3.1	Complementary Interpretability Strategies	75
5.3.2	The Polysemanticity-Monosemanticity Trade-off	76
5.3.3	Implications for LLM Interpretability	76
5.4	Limitations and Future Directions	76
5.4.1	Current Limitations	76
5.4.2	Future Research Directions	77
5.5	Broader Implications	78
5.5.1	For AI Safety	78
5.5.2	For Machine Learning Theory	78
6	Conclusion	79
6.1	Thesis Summary	79
6.2	Core Contributions	79
6.3	Resolving the Central Question	80
6.4	Broader Significance	80
6.5	Final Reflection	80

List of Figures

2.1	Sparse Autoencoder architecture showing the flow from input through encoder to sparse features and decoder to reconstruction.	13
2.2	Self-attention architecture. The feature dimension is tied to batch size N	16
2.3	Side-by-side comparison of SAE and ST architectures. SAE uses feedforward encoding with ReLU activation, while ST uses attention-based feature computation with softmax normalization.	19
3.1	Cross-attention architecture. The value projection $\mathbf{W}_V$ scales with input dimension.	28
3.2	Parameterized key-value attention architecture. Keys and values are learned parameters.	29
4.1	Effect of varying feature dimension on model performance measured by cosine distance for MNIST dataset. Boxes are slightly shifted for visibility but correspond to their nearest feature dimension.	37
4.2	Effect of varying feature dimension on model performance measured by cosine distance for Fashion-MNIST dataset. Boxes are slightly shifted for visibility but correspond to their nearest feature dimension.	38
4.3	Effect of varying feature dimension on model performance measured by Euclidean distance for MNIST dataset. The logarithmic y-axis highlights the dramatic performance difference, with SAE models achieving centroid distances 100–1000 times larger than ST models.	39
4.4	Effect of varying feature dimension on model performance measured by Euclidean distance for Fashion-MNIST dataset. The logarithmic y-axis reveals SAE models achieving 100–1000 $\times$ greater separation than ST models.	40
4.5	Impact of different L1 regularization strengths on model performance using cosine distance metric. MNIST dataset. . . .	41

4.6	Impact of different L1 regularization strengths on model performance using cosine distance metric. Fashion-MNIST dataset.	42
4.7	Impact of different L1 regularization strengths on model performance using Euclidean distance metric. MNIST dataset.	42
4.8	Impact of different L1 regularization strengths on model performance using Euclidean distance metric. Fashion-MNIST dataset.	43
4.9	Effect of varying learning rates on model performance measured by cosine distance. MNIST dataset.	44
4.10	Effect of varying learning rates on model performance measured by cosine distance. Fashion-MNIST dataset.	44
4.11	Effect of varying learning rates on model performance measured by Euclidean distance. MNIST dataset.	45
4.12	Effect of varying learning rates on model performance measured by Euclidean distance. Fashion-MNIST dataset.	45
4.13	Feature importance distribution for SAE with $f = 400$ on MNIST.	47
4.14	Feature importance heatmap for 400D SAE showing top 5 rankings per class. Numbers indicate rank (1-5), with darker shading for higher ranks. Dash indicates feature not in top 5. Shows 25 most important features across all classes.	47
4.15	Feature importance distribution for SAE with $f = 1600$ on MNIST.	48
4.16	Feature importance heatmap for 1600D SAE showing top-ranked features per class. Most features appear for single classes only. Numbers indicate rank (1-5), with darker shading for higher ranks.	48
4.17	Top 5 most important features for the digit 3 with SAE 400D model.	49
4.18	Top 5 most important features for the digit 3 with SAE 1600D model.	49
4.19	Feature interaction grid for SAE model using 400-dimensional latent space (features 142 and 210).	50
4.20	Feature interaction grid for SAE model using 1600-dimensional latent space (features 973 and 1539).	51
4.21	Feature importance distribution for ST with $f = 400$ on MNIST.	52
4.22	Feature importance heatmap for 400D ST showing top 5 rankings per class. Complete specialization with no feature sharing across classes. Numbers indicate rank, with darker shading for higher ranks.	52
4.23	Feature importance distribution for ST with $f = 1600$ on MNIST.	53

4.24 Feature importance heatmap for 1600D ST showing top-ranked features per class. Complete specialization maintained with no feature sharing. Numbers indicate rank, with darker shading for higher ranks.	54
4.25 Top 5 most important features for the digit 3 with ST 400D model.	55
4.26 Top 5 most important features for the digit 3 with ST 1600D model.	55
4.27 Feature interaction grid for ST model using 400-dimensional latent space (features 378 and 39).	56
4.28 Feature interaction grid for ST model using 1600-dimensional latent space (features 857 and 409).	56
4.29 Feature importance distribution for SAE with $f=400$ on Fashion-MNIST.	57
4.30 Feature importance heatmap for 400D SAE showing top 5 rankings per class. Complete specialization with no feature sharing across classes. Numbers indicate rank, with darker shading for higher ranks.	57
4.31 Feature importance distribution for SAE with $f=1600$ on Fashion-MNIST.	58
4.32 Feature importance heatmap for 1600D SAE showing top-ranked features per class. Limited sharing emerges: F58 and F1316 shared by classes 4 and 6; F94 shared by classes 5 and 7. Numbers indicate rank, with darker shading for higher ranks.	59
4.33 Feature interaction grid for SAE model on Fashion-MNIST using 400-dimensional latent space (features 191 and 56).	60
4.34 Feature interaction grid for SAE model on Fashion-MNIST using 1600-dimensional latent space (features 1099 and 1253).	60
4.35 Feature importance distribution for ST with $f = 400$ on Fashion-MNIST.	61
4.36 Feature importance heatmap for 400D ST showing top 5 rankings per class. Near-complete specialization with only F93 shared between classes 4 and 6. Numbers indicate rank, with darker shading for higher ranks.	61
4.37 Feature importance distribution for ST with $f = 1600$ on Fashion-MNIST.	62
4.38 Feature importance heatmap for 1600D ST showing top-ranked features per class. Complete specialization with no feature sharing. Numbers indicate rank, with darker shading for higher ranks.	63
4.39 Feature interaction grid for ST model (400D). Features 59 and 172.	64
4.40 Feature interaction grid for ST model (1600D). Features 1596 and 1453.	64

4.41 Average centroid distance by layer using Euclidean metric on GPT-Neo 1.3B representations.	67
4.42 Average centroid distance by layer using cosine metric on GPT-Neo 1.3B representations.	68

List of Tables

2.1	Comparison of sparsity mechanisms between architectures	20
2.2	Interpretability trade-offs between architectures	21
4.1	Average centroid distances by layer for both architectures on GPT-Neo 1.3B	69
4.2	Performance gap comparison between toy datasets and LLM representations	70



Introduction

1.1 Motivation

As neural networks achieve unprecedented capabilities across domains from computer vision to natural language processing, their internal mechanisms remain fundamentally opaque. This opacity, often characterized as the “black box” problem, poses critical challenges for AI safety, reliability, and trust. When a language model generates harmful content or a medical diagnostic system makes an error, we lack the tools to understand why these failures occurred or how to prevent them systematically. The field of mechanistic interpretability has emerged to address this challenge, seeking to reverse-engineer the computational structures within neural networks to understand how they process information and make decisions.

At the heart of the interpretability challenge lies *polysemanticity*: the phenomenon where individual neurons respond to multiple, semantically unrelated concepts. In a polysemantic network, a single neuron might activate for both “dogs” and “cars,” making it impossible to assign clear semantic meaning to individual components. This distributed representation, while computationally efficient for the network, creates a fundamental barrier to understanding. We cannot simply inspect neurons to determine what features a network has learned or trace how information flows through the system.

1.2 Sparse Representations

Sparse representation learning offers a compelling solution to polysemanticity. By learning overcomplete dictionaries where the number of features exceeds the input dimension, sparse methods can potentially disentangle superposed concepts into interpretable components. The key insight is that while neural networks may represent many concepts in a compressed, entangled form for efficiency, we can learn a sparse “dictionary” that separates these concepts into distinct, interpretable features.

One prominent approaches have emerged for learning sparse representations of neural network activations: the Sparse Autoencoder (SAE). We propose a new architecture: the Sparse Transformer (ST). Both architectures aim to decompose neural network activations into sparse, interpretable features, but they achieve sparsity through fundamentally different mechanisms. SAEs employ ReLU activations and L1 regularization to create exact sparsity, where features are either fully active or completely silent. In contrast, STs leverage attention mechanisms with softmax normalization to create competitive dynamics between features accelerated by L1 regularization, resulting in approximate sparsity through probability concentration.

1.3 Research Question

This thesis addresses a fundamental question: *Do these architectural differences merely represent implementation details, or do they lead to fundamentally different solutions to the interpretability problem?* We hypothesize that the choice of sparsity mechanism, thresholding versus competition, creates distinct geometric biases in the learned representations, leading to complementary strengths and weaknesses.

1.4 Thesis Contributions

This thesis makes three primary contributions to the field of neural network interpretability:

1. **Empirical Validation:** Through systematic experiments on visual datasets (MNIST and Fashion-MNIST), we demonstrate that these theoretical differences manifest as dramatic performance gaps: SAEs achieve 100–1000× superior Euclidean separation while STs show 10× better angular discrimination.

2. **Feature-Level Analysis:** We provide a detailed analysis of how individual features contribute to class discrimination in both architectures, revealing that SAEs learn shared, polysemantic features while STs develop highly specialized, monosemantic representations even at low capacity.
3. **Scalability Assessment:** By applying both architectures to GPT-Neo 1.3B representations, we show that while geometric biases persist at scale, performance gaps narrow substantially (from 100–1000 $\times$ to 3–30 $\times$), suggesting that architectural choice becomes less critical for practical applications.

1.5 Thesis Structure

This thesis is organized as follows:

Chapter 2 establishes the theoretical foundations, beginning with the broader context of neural network interpretability and the polysemanticity problem. We develop mathematical frameworks for both SAEs and STs, analyzing their optimization objectives, sparsity mechanisms, and induced geometric properties. The chapter concludes with a comparative analysis predicting how architectural differences should manifest in practice.

Chapter 3 presents our experimental methodology, detailing the datasets, model implementations, hyperparameter search strategies, and evaluation metrics. We introduce novel approaches for feature-level analysis, including importance quantification and systematic feature manipulation protocols.

Chapter 4 reports comprehensive empirical results across three experimental domains. We validate theoretical predictions through hyperparameter analysis, reveal unexpected phenomena like noise-based encoding in STs, and demonstrate how architectural biases persist but diminish when applied to large language model representations.

Chapter 5 synthesizes our findings and examines their implications for mechanistic interpretability. We discuss the complementary nature of SAEs and STs, the polysemanticity-monosemanticity trade-off, and limitations of current approaches. The chapter concludes by outlining promising directions for future research, including hybrid architectures and theoretical extensions.

Chapter 6 concludes the thesis, reflecting on the broader significance of our geometric framework for understanding neural network interpretability methods.

Part I

Background

/2

Theoretical Foundations

2.1 Introduction

As neural networks become increasingly central to critical applications, from medical diagnosis to autonomous vehicles to large language models shaping public discourse, the inability to understand their internal mechanisms poses fundamental risks to safety, reliability, and trust. This chapter establishes the theoretical foundations for addressing neural network interpretability through sparse representation learning, focusing on two competing models: Sparse Autoencoders (SAEs) and Sparse Transformers (STs).

The core challenge we address is polysemanticity: the phenomenon where individual neurons in neural networks respond to multiple, semantically unrelated concepts. This distributed representation, while computationally efficient, creates an interpretability barrier: we cannot simply inspect individual neurons to understand what features a network has learned. Both SAEs and STs attempt to solve this problem by learning sparse, overcomplete representations that disentangle these superposed features into interpretable components.

This chapter develops the mathematical frameworks necessary to understand how these architectures achieve sparsity through fundamentally different mechanisms. SAEs employ ReLU activations and L1 regularization to create exact sparsity, where features are either fully active or completely silent. In contrast, STs leverage attention mechanisms with softmax normalization to create competitive dynamics between features, resulting in approximate sparsity through

probability concentration. These architectural choices lead to profound differences in the geometric properties of learned representations, differences that manifest as the complementary strengths revealed in our empirical results.

We begin by examining the broader context of neural network interpretability (Section 2.2), establishing why traditional approaches fail for modern deep networks and motivating the need for specialized interpretability methods. Section 2.3 provides the mathematical foundations of sparse dictionary learning and formally characterizes the polysemanticity problem that both architectures aim to solve. Given the particular importance of interpretability for large language models, Section 2.4 examines the unique challenges posed by transformer-based architectures and their high-dimensional representations.

The chapter then develops comprehensive theoretical analyses of both architectures. Section 2.5 presents the mathematical formulation of Sparse Autoencoders, analyzing their optimization objectives, induced geometric properties, and the mechanism by which ReLU activations create interpretable on/off feature behavior. Section 2.6 introduces our novel Sparse Transformer architecture, demonstrating how attention mechanisms can be repurposed for sparse feature learning and analyzing the resulting competitive dynamics.

Finally, Section 2.7 provides a unified comparative framework, revealing how the architectural differences between SAEs and STs lead to complementary geometric biases: SAEs naturally optimize for Euclidean separation through unbounded activations, while STs excel at angular separation through their normalized attention weights.

By establishing these theoretical foundations, this chapter provides the conceptual tools necessary to understand not just what each architecture achieves, but why their different approaches to sparsity lead to fundamentally different solutions to the interpretability problem.

2.2 Neural Network Interpretability

2.2.1 The Black Box Problem

Modern neural networks achieve remarkable performance but remain fundamentally opaque in their decision-making processes. This opacity stems from several interconnected challenges that make interpretability a central concern in AI safety and deployment [24, 9].

The interpretability challenges manifest at multiple levels:

1. **Distributed representations:** Neural networks encode information across many neurons simultaneously, preventing simple feature-to-neuron mappings [27]
2. **Non-linear transformations:** The composition of non-linear activation functions creates complex decision boundaries that resist linear analysis [19]
3. **High dimensionality:** Modern architectures contain billions of parameters, making exhaustive analysis computationally infeasible [5]
4. **Hierarchical processing:** Features emerge at multiple levels of abstraction, with complex interactions between layers [30]

2.2.2 Approaches to Interpretability

The field has developed two complementary approaches to address these challenges [24]:

Intrinsic Interpretability

These methods build transparency directly into model architecture:

- **Linear models:** Provide explicit feature weights but limited expressiveness
- **Decision trees:** Offer readable decision paths but poor generalization
- **Attention mechanisms:** Enable inspection of input-output relationships [2]
- **Sparse models:** Our focus, which enforce interpretable feature selection through architectural constraints

Post-hoc Explanation Methods

These techniques analyze trained models without modifying their architecture:

- **Feature attribution:** Methods like SHAP [17] and LIME [25] assign

importance scores to inputs

- **Concept activation vectors:** Identify directions in activation space corresponding to human concepts [15]
- **Mechanistic interpretability:** Reverse-engineer computational circuits within networks [10, 20]

This thesis focuses on intrinsic interpretability through sparse architectures, which we argue provides more reliable insights than post-hoc analysis.

2.3 Sparse Dictionary Learning and Polysemanticity

2.3.1 Mathematical Foundations of Sparse Coding

Sparse dictionary learning provides the theoretical foundation for understanding sparse representations in neural networks [22]. Given a dataset $\mathcal{X} = \{\mathbf{x}_i\}_{i=1}^N$ where $\mathbf{x}_i \in \mathbb{R}^n$, the goal is to find a dictionary $\mathbf{D} \in \mathbb{R}^{n \times m}$ and sparse codes $\{\boldsymbol{\alpha}_i\}_{i=1}^N$ where $\boldsymbol{\alpha}_i \in \mathbb{R}^m$ such that:

$$\min_{\mathbf{D}, \{\boldsymbol{\alpha}_i\}} \sum_{i=1}^N [\|\mathbf{x}_i - \mathbf{D}\boldsymbol{\alpha}_i\|_2^2 + \lambda \|\boldsymbol{\alpha}_i\|_0] \quad (2.1)$$

The ℓ_0 pseudo-norm $\|\boldsymbol{\alpha}_i\|_0$ counts non-zero elements, enforcing sparsity. Due to NP-hardness of ℓ_0 optimization, practical algorithms use the ℓ_1 relaxation:

$$\min_{\mathbf{D}, \{\boldsymbol{\alpha}_i\}} \sum_{i=1}^N [\|\mathbf{x}_i - \mathbf{D}\boldsymbol{\alpha}_i\|_2^2 + \lambda \|\boldsymbol{\alpha}_i\|_1] \quad (2.2)$$

This formulation has deep connections to compressed sensing theory [6] and provides theoretical guarantees on reconstruction quality under appropriate conditions on dictionary coherence and signal sparsity [8].

2.3.2 Polysemanticity in Neural Networks

Polysemanticity, the phenomenon where individual neurons respond to multiple, semantically unrelated features, represents a fundamental challenge to neural network interpretability [11, 21, 7, 4].

Theoretical Model of Polysemanticity

Consider a neural network layer with n neurons tasked with representing $m > n$ features. Let $\mathbf{f} \in \{0, 1\}^m$ be a binary feature vector and $\mathbf{h} \in \mathbb{R}^n$ be the neural activations. The network must learn an encoding function $E : \{0, 1\}^m \rightarrow \mathbb{R}^n$ and decoding function $D : \mathbb{R}^n \rightarrow \mathbb{R}^m$.

Under the superposition hypothesis [11], the network represents features using non-orthogonal directions:

$$\mathbf{h} = \sum_{i=1}^m f_i \mathbf{w}_i + \epsilon \quad (2.3)$$

where $\mathbf{w}_i \in \mathbb{R}^n$ are feature directions and ϵ represents interference noise. The expected interference between features i and j is:

$$\mathbb{E}[\text{interference}_{ij}] = \mathbb{E}[f_i f_j] \langle \mathbf{w}_i, \mathbf{w}_j \rangle \quad (2.4)$$

When features are sparse ($\mathbb{P}(f_i = 1) \ll 1$), the probability of simultaneous activation $\mathbb{E}[f_i f_j]$ becomes negligible for $i \neq j$, allowing the network to pack many nearly-orthogonal features into a lower-dimensional space. This superposition hypothesis has been extensively studied in recent work [11, 26, 7].

Implications for Interpretability

Polysemanticity creates three critical challenges:

1. **Feature attribution ambiguity:** A single neuron's activation cannot be uniquely attributed to one semantic concept
2. **Circuit analysis complexity:** Tracing information flow requires disentangling superposed features

3. **Intervention unreliability:** Modifying individual neurons affects multiple unrelated features

2.3.3 Sparse Representations as a Solution

Both SAEs and STs attempt to address polysemanticity by learning sparse, overcomplete representations where $m > n$. This creates sufficient capacity for monosemantic features while maintaining efficiency through sparsity. However, as we will demonstrate, their different architectural choices lead to fundamentally different solutions to this problem.

2.4 Large Language Models and the Need for Interpretability

2.4.1 Transformer Architecture Fundamentals

Large Language Models build upon the transformer architecture [29], which processes sequences through self-attention mechanisms. For a sequence $\mathbf{X} \in \mathbb{R}^{L \times d}$ where L is sequence length and d is model dimension:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V} \quad (2.5)$$

where queries $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$, keys $\mathbf{K} = \mathbf{X}\mathbf{W}_K$, and values $\mathbf{V} = \mathbf{X}\mathbf{W}_V$ are linear projections of the input.

2.4.2 The Need for Interpretability

As LLMs are deployed in critical applications, understanding their internal representations becomes essential for:

1. **Safety assurance:** Detecting and mitigating harmful behaviors before deployment [1, 14]
2. **Capability assessment:** Understanding what models truly "know" versus memorize [18]
3. **Debugging and improvement:** Identifying failure modes and optimization

tion targets [20]

Recent work has shown that LLM activations exhibit severe polysemanticity [12, 28], motivating the development of specialized interpretability methods.

2.5 Sparse Autoencoders: Theory and Architecture

2.5.1 Architectural Formulation

Sparse Autoencoders extend classical autoencoders by introducing sparsity constraints and overcomplete representations [7, 4]. For an input $\mathbf{x} \in \mathbb{R}^n$, the SAE learns:

$$\begin{aligned} \text{Encoder : } \mathbf{f}(\mathbf{x}) &= \sigma(\mathbf{W}_e \mathbf{x} + \mathbf{b}_e) \\ \text{Decoder : } \hat{\mathbf{x}} &= \mathbf{W}_d \mathbf{f}(\mathbf{x}) + \mathbf{b}_d \end{aligned} \quad (2.6)$$

where $\mathbf{W}_e \in \mathbb{R}^{m \times n}$, $\mathbf{W}_d \in \mathbb{R}^{n \times m}$ with $m > n$, and σ is typically ReLU.

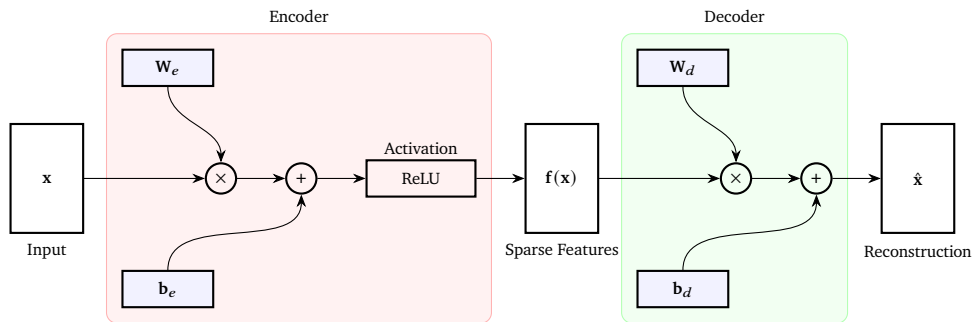


Figure 2.1: Sparse Autoencoder architecture showing the flow from input through encoder to sparse features and decoder to reconstruction.

2.5.2 Optimization Objective

The SAE loss function balances reconstruction accuracy with sparsity:

$$\mathcal{L}_{\text{SAE}} = \underbrace{\frac{1}{N} \sum_{i=1}^N \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|_2^2}_{\text{Reconstruction}} + \underbrace{\lambda \sum_{j=1}^m \mathbb{E}[\|f_j\|] \|\mathbf{w}_{d,j}\|_2}_{\text{Sparsity}} \quad (2.7)$$

The decoder normalization $\|\mathbf{w}_{d,j}\|_2$ prevents the trivial solution of scaling encoder weights up and decoder weights down [4, 23].

2.5.3 Geometric Properties and Sparsity Mechanism

Induced Geometry

The ReLU activation creates a piecewise linear mapping that partitions the input space into 2^m regions. Within each region, the transformation is linear:

$$\mathbf{f}(\mathbf{x}) = \text{diag}(\mathbf{s})(\mathbf{W}_e \mathbf{x} + \mathbf{b}_e) \quad (2.8)$$

where $\mathbf{s} \in \{0, 1\}^m$ indicates which features are active.

Sparsity Through Thresholding

The L1 penalty combined with ReLU creates a soft threshold effect. For each feature j , activation occurs when:

$$\mathbf{w}_{e,j}^T \mathbf{x} + b_{e,j} > \tau_j \quad \text{where} \quad \tau_j \propto \frac{\lambda}{\|\mathbf{w}_{d,j}\|_2} \quad (2.9)$$

This creates binary on/off behavior, making features interpretable as discrete concepts.

2.5.4 Theoretical Advantages and Limitations

Recent work has demonstrated these properties empirically in large-scale applications [28, 13].

Advantages:

- **Exact sparsity:** ReLU creates true zeros, enabling clear feature attribution
- **Unbounded activations:** Can represent arbitrarily strong feature presence
- **Simple optimization:** Piecewise linear structure amenable to gradient descent

Limitations:

- **Dead features:** Neurons with negative pre-activations receive no gradients
- **Discrete transitions:** Discontinuous gradients at ReLU boundaries
- **Magnitude sensitivity:** Performance depends on input scaling

2.6 Sparse Transformers: Theoretical Foundations

2.6.1 Motivation for Attention-Based Sparsity

We propose leveraging attention mechanisms for sparse representations, motivated by:

1. **Architectural alignment:** Since transformers are the foundation of modern LLMs, attention-based interpretability may provide more direct insights
2. **Normalization properties:** Attention weights naturally sum to one, providing automatic regularization
3. **Smooth optimization:** Softmax provides gradients everywhere, avoiding dead features
4. **Relational encoding:** Attention naturally captures relationships between features

2.6.2 Theoretical Architecture

For theoretical analysis, we consider an attention-based sparse representation where features are computed as:

$$\mathbf{f}(\mathbf{x}) = \text{softmax} \left(\frac{\mathbf{Q}(\mathbf{x})\mathbf{K}^T}{\sqrt{d_k}} \right) \quad (2.10)$$

where $\mathbf{Q}(\mathbf{x})$ is a query projection and $\mathbf{K}$ represents learned key vectors. The reconstruction is:

$$\hat{\mathbf{x}} = \mathbf{f}(\mathbf{x})\mathbf{V} \quad (2.11)$$

where $\mathbf{V}$ represents learned value vectors. The specific parameterization of these components is detailed in the Methodology chapter.

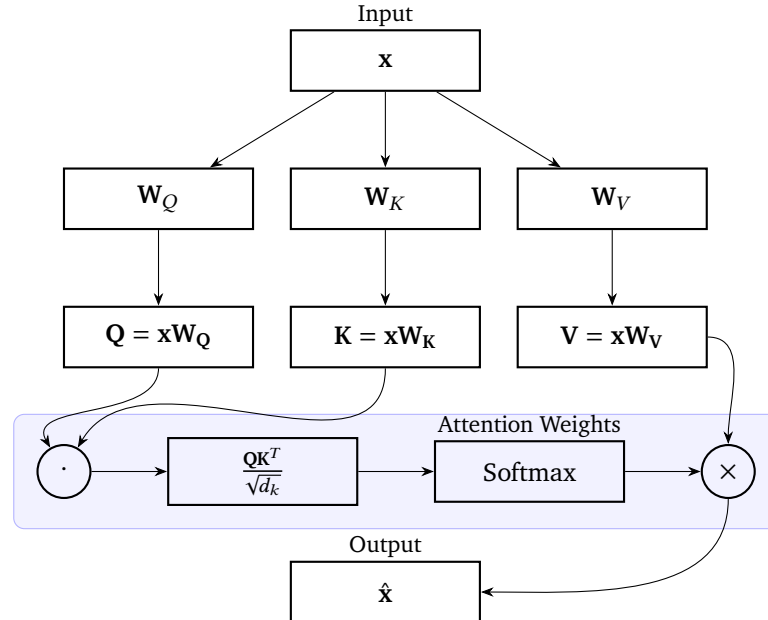


Figure 2.2: Self-attention architecture. The feature dimension is tied to batch size N .

2.6.3 Sparsity Mechanism in Sparse Transformers

Softmax-Induced Sparsity

The softmax function creates a fundamentally different sparsity mechanism:

$$f_i = \frac{\exp(z_i)}{\sum_{j=1}^m \exp(z_j)}, \quad \text{where } z_i = \frac{\mathbf{q} \cdot \mathbf{k}_i}{\sqrt{d_k}} \quad (2.12)$$

This creates:

- **No exact zeros:** $f_i > 0$ for all i , though values can be arbitrarily small
- **Competition constraint:** $\sum_{i=1}^m f_i = 1$ forces features to compete
- **Temperature control:** The scaling factor $\sqrt{d_k}$ controls distribution sharpness

Competitive Dynamics

The gradient structure reveals the competitive mechanism:

$$\frac{\partial f_i}{\partial z_j} = f_i(\delta_{ij} - f_j) \quad (2.13)$$

Increasing z_i not only increases f_i but decreases all other features proportionally, creating winner-take-all dynamics.

Effective Sparsity Through L1 Regularization

Under normalization constraint $\sum_i f_i = 1$, L1 regularization encourages entropy minimization:

$$\mathcal{L}_{\text{ST}} = \mathbb{E}_{\mathbf{x}} [\|\mathbf{x} - \hat{\mathbf{x}}\|_2^2] + \lambda \sum_{i=1}^m \mathbb{E}_{\mathbf{x}}[f_i] \|\mathbf{v}_i\|_2 \quad (2.14)$$

This drives the attention distribution toward lower entropy, concentrating probability mass on fewer features.

2.6.4 Theoretical Properties

Angular Optimization

The dot product attention mechanism emphasizes angular relationships:

$$z_i = \frac{\|\mathbf{q}\| \|\mathbf{k}_i\|}{\sqrt{d_k}} \cos(\theta_{qk_i}) \quad (2.15)$$

This creates a natural bias toward angular rather than Euclidean separation in feature space.

Gradient Stability

Unlike SAE's discontinuous ReLU gradients, ST gradients are always well-defined:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{k}_i} = -\mathbb{E}_{\mathbf{x}}[(\mathbf{x} - \hat{\mathbf{x}})^T \mathbf{v}_i f_i(1 - f_i) \frac{\mathbf{q}}{\sqrt{d_k}}] \quad (2.16)$$

The term $f_i(1 - f_i) > 0$ ensures all features receive gradient updates.

2.7 Comparative Theoretical Analysis

2.7.1 Unified Framework

Both architectures (Figure 2.3) solve the sparse coding problem with different constraints:

$$\min_{\theta} \mathbb{E}_{\mathbf{x}}[\|\mathbf{x} - \mathbf{D}\mathbf{f}_{\theta}(\mathbf{x})\|_2^2] + \lambda \Omega(\mathbf{f}_{\theta}(\mathbf{x})) \quad (2.17)$$

where:

- SAE: $\mathbf{f}_{\theta}(\mathbf{x}) = \text{ReLU}(\mathbf{W}_e \mathbf{x} + \mathbf{b}_e)$, $\Omega(\mathbf{f}) = \sum_i |f_i| \|\mathbf{w}_{d,i}\|_2$, $\mathbf{D} = \mathbf{W}_d$
- ST: $\mathbf{f}_{\theta}(\mathbf{x}) = \text{softmax}(\mathbf{Q}(\mathbf{x})\mathbf{K}^T / \sqrt{d_k})$, $\Omega(\mathbf{f}) = \sum_i |f_i| \|\mathbf{v}_i\|_2$, $\mathbf{D} = \mathbf{V}$

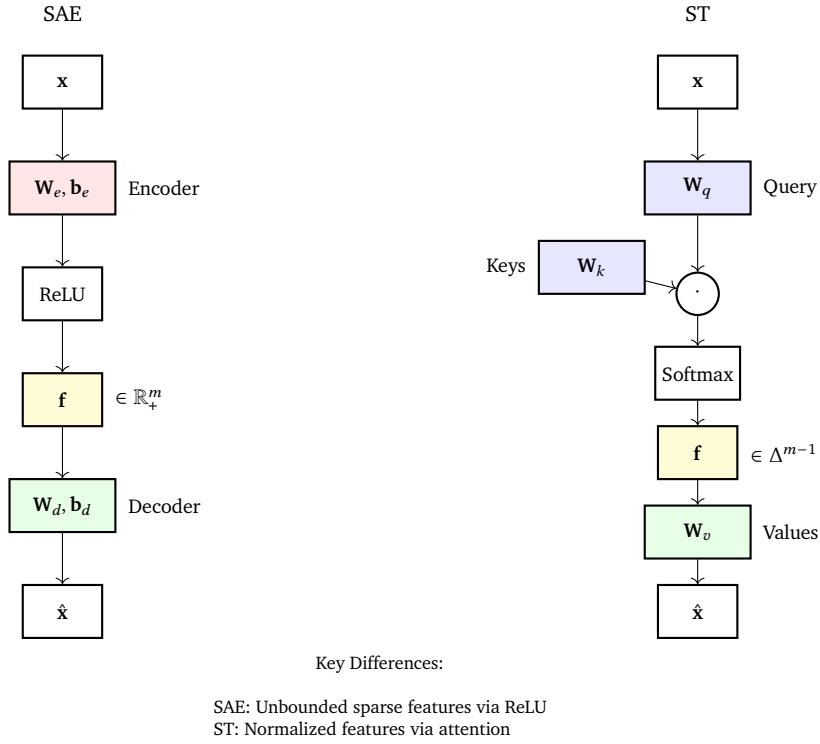


Figure 2.3: Side-by-side comparison of SAE and ST architectures. SAE uses feedforward encoding with ReLU activation, while ST uses attention-based feature computation with softmax normalization.

2.7.2 Geometric Comparison

Feature Manifolds

The architectures constrain features to different geometric spaces:

- **SAE:** Features lie in $\mathbb{R}_+^m$ (positive orthant)
 - Unbounded activation magnitudes
 - Independent feature activation
- **ST:** Features lie on the $(m - 1)$ -simplex $\Delta^{m-1} = \{\mathbf{f} \in \mathbb{R}_+^m : \sum_i f_i = 1\}$
 - Bounded activation magnitudes
 - Competitive feature activation

Distance Properties

For features $\mathbf{f}_1, \mathbf{f}_2$ from inputs $\mathbf{x}_1, \mathbf{x}_2$:

SAE: Unbounded Euclidean distance

$$d_{\text{SAE}}^E = \|\mathbf{f}_1 - \mathbf{f}_2\|_2 \in [0, \infty) \quad (2.18)$$

ST: Bounded Euclidean distance

$$d_{\text{ST}}^E = \|\mathbf{f}_1 - \mathbf{f}_2\|_2 \in [0, \sqrt{2}] \quad (2.19)$$

The maximum ST distance occurs when $\mathbf{f}_1$ and $\mathbf{f}_2$ are one-hot vectors with different active indices.

2.7.3 Sparsity Mechanism Comparison

Property	SAE (ReLU)	ST (Softmax)
Zero values	Exact zeros	Approximate zeros
Sparsity control	Direct thresholding	Competitive suppression
Feature independence	Independent activation	Mutually exclusive
Gradient behavior	Discontinuous at zero	Smooth everywhere
Total activation	Unbounded	Normalized to 1

Table 2.1: Comparison of sparsity mechanisms between architectures

2.7.4 Optimization Landscape Analysis

Convergence Properties

The different gradient structures lead to distinct optimization behaviors:

- **SAE:** Piecewise linear landscape with potential for dead features
- **ST:** Smooth landscape with guaranteed gradient flow to all features

Learning Dynamics

The architectural differences manifest in practice as:

- **SAE:** Requires careful initialization to avoid dead features; benefits from higher learning rates

- **ST:** Shows robust convergence across hyperparameter settings; stable with conservative learning rates

2.7.5 Interpretability Trade-offs

The fundamental trade-offs between architectures:

Aspect	SAE	ST
Feature clarity	Binary on/off	Graded activation
Geometric bias	Euclidean separation	Angular separation
Capacity utilization	Variable (dead features)	Full (all features active)
Optimization stability	Sensitive to initialization	Robust convergence
Magnitude interpretation	Absolute strength	Relative importance

Table 2.2: Interpretability trade-offs between architectures

2.7.6 Theoretical Predictions

Based on this analysis, we predict:

1. **Euclidean superiority of SAE:** Unbounded activations should enable greater Euclidean separation
2. **Angular superiority of ST:** Normalization and competition should enhance angular discrimination
3. **Dataset complexity effects:** Simple datasets (MNIST) may show more extreme separation differences than complex datasets (Fashion-MNIST)
4. **Scaling behavior:** Performance gaps may diminish when applied to high-dimensional LLM representations

These predictions are empirically validated in subsequent chapters.

Part II

Methodology

/3

Methodology

3.1 Introduction

This chapter presents the experimental framework for comparing Sparse Autoencoders (SAEs) and Sparse Transformers (STs) through systematic evaluation of their geometric representation properties. We design experiments that reveal not only performance differences but also the underlying mechanisms driving these differences. Our methodology encompasses three experimental domains: controlled evaluation on visual datasets (MNIST and Fashion-MNIST), detailed feature-level analysis, and validation on large language model representations from GPT-Neo 1.3B.

3.2 Experimental Framework

3.2.1 Dataset Selection

We selected three experimental domains to comprehensively evaluate architectural differences:

Visual Datasets

- **MNIST:** Handwritten digits providing simple geometric patterns with clear class boundaries
- **Fashion-MNIST:** Clothing items offering increased complexity through textures and subtle inter-class differences

These datasets enable controlled experimentation with known ground truth while providing a complexity gradient. MNIST’s simplicity (sparse pixel patterns, high inter-class separation) contrasts with Fashion-MNIST’s challenges (dense textures, low inter-class separation), allowing us to observe how architectural differences manifest across complexity levels.

Language Model Representations

- **GPT-Neo 1.3B:** Intermediate layer representations (layers 12-16) with 2048-dimensional activations
- **Feature dimension:** $f = 8000$ to match typical SAE configurations for LLMs

This validates whether patterns observed in controlled settings generalize to high-dimensional, real-world representations.

3.2.2 Evaluation Metrics

We employ unsupervised metrics that directly measure representation quality without relying on downstream task performance:

Centroid Distance Analysis

For each trained model, we compute class centroids in the learned feature space:

$$\mathbf{c}_k = \frac{1}{|\mathcal{X}_k|} \sum_{\mathbf{x} \in \mathcal{X}_k} \mathbf{f}(\mathbf{x}) \quad (3.1)$$

We then measure separation using two complementary metrics:

- **Normalized Euclidean Distance:** $d_{\text{euclidean}}(\mathbf{c}_i, \mathbf{c}_j) = \frac{\|\mathbf{c}_i - \mathbf{c}_j\|_2}{\sqrt{m}}$
- **Cosine Distance:** $d_{\text{cosine}}(\mathbf{c}_i, \mathbf{c}_j) = 1 - \frac{\mathbf{c}_i \cdot \mathbf{c}_j}{\|\mathbf{c}_i\|_2 \|\mathbf{c}_j\|_2}$

Feature Interpretability Analysis

- **Feature Importance:** Random Forest-based importance scores for global and class-specific relevance
- **Feature Manipulation:** Systematic activation of individual features to observe reconstruction patterns
- **Feature Interaction:** 2D grids varying pairs of features to identify compositional behavior

3.3 Model Implementations

3.3.1 Sparse Autoencoder Configuration

Following the theoretical framework from Chapter 2, our SAE implementation uses:

- **Architecture:** $\mathbf{f}(\mathbf{x}) = \text{ReLU}(\mathbf{W}_e \mathbf{x} + \mathbf{b}_e)$, $\hat{\mathbf{x}} = \mathbf{W}_d \mathbf{f}(\mathbf{x}) + \mathbf{b}_d$
- **Loss Function:** $\mathcal{L}_{\text{SAE}} = \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2 + \lambda \sum_{j=1}^m |f_j| \|\mathbf{w}_{d,j}\|_2$
- **Decoder Normalization:** L2 normalization applied to decoder columns after each update
- **Initialization:** Xavier uniform for weights, zeros for biases

3.3.2 Sparse Transformer Implementation

Architectural Challenge

Standard attention mechanisms [29] face fundamental limitations for sparse dictionary learning:

- **Self-attention:** Feature count tied to batch size, preventing fixed dictionary size
- **Cross-attention:** Requires prohibitive $\mathbf{W}_V \in \mathbb{R}^{n \times n}$ scaling quadratically with input dimension

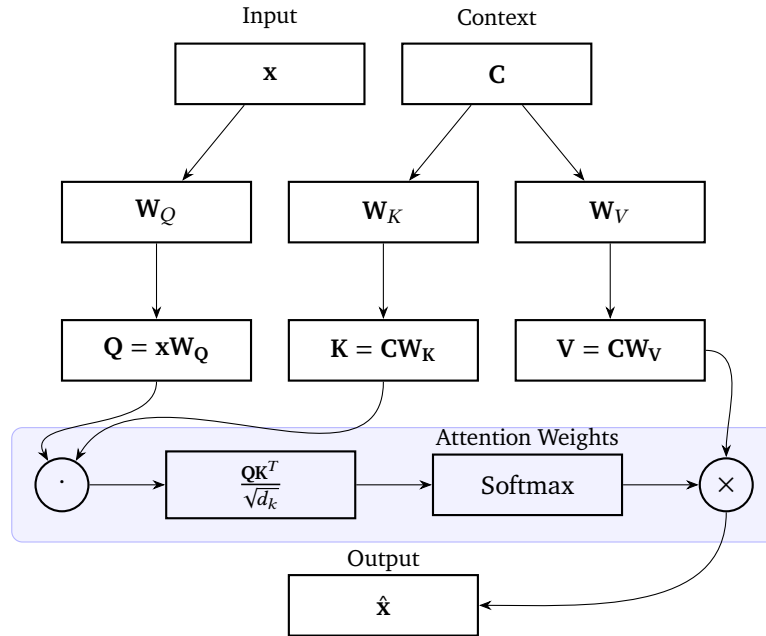


Figure 3.1: Cross-attention architecture. The value projection $\mathbf{W}_V$ scales with input dimension.

Parameterized Key-Value Solution

We directly parameterize keys and values as learned matrices:

$$\begin{aligned}
 \text{Query projection : } \mathbf{Q} &= \mathbf{x}\mathbf{W}_q, \quad \mathbf{W}_q \in \mathbb{R}^{n \times a} \\
 \text{Keys : } \mathbf{K} &= \mathbf{W}_k \in \mathbb{R}^{m \times a} \\
 \text{Values : } \mathbf{V} &= \mathbf{W}_v \in \mathbb{R}^{m \times n}
 \end{aligned} \tag{3.2}$$

This yields sparse features through attention:

$$\mathbf{f}(\mathbf{x}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{a}}\right) \quad (3.3)$$

And reconstruction:

$$\hat{\mathbf{x}} = \mathbf{f}(\mathbf{x})\mathbf{V} \quad (3.4)$$

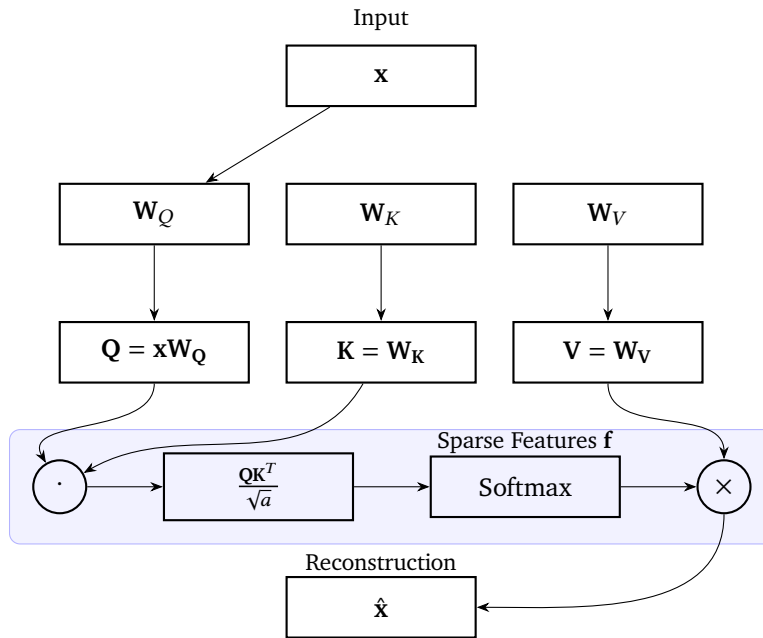


Figure 3.2: Parameterized key-value attention architecture. Keys and values are learned parameters.

Parameter Matching

To ensure fair comparison, we set the attention dimension a to match SAE parameter count:

$$a = \frac{nm + m + n}{n + m} \quad (3.5)$$

This typically results in $a \ll n$, making ST more parameter-efficient in its attention mechanism.

3.4 Hyperparameter Grid

3.4.1 Systematic Exploration

We conduct exhaustive grid search over:

- **Feature dimensions:** $m \in \{20, 50, 100, 200, 400, 800, 1600\}$
- **L1 regularization:** $\lambda \in \{2.5, 5.0, 10.0\}$
- **Learning rates:** $\eta \in \{5 \times 10^{-5}, 1 \times 10^{-3}\}$

3.4.2 Adaptive Training Schedule

Training steps scale with model complexity:

$$\text{steps} = 200000 \cdot \left(\frac{m/n}{8} \right)^{0.75} \quad (3.6)$$

This ensures adequate training for larger models while maintaining computational efficiency.

3.4.3 Fixed Parameters

- **Batch size:** 4096 samples
- **Optimizer:** Adam [16] with default parameters
- **Device:** CUDA-enabled GPU

3.5 Feature Analysis Methodology

3.5.1 Feature Importance Quantification

We employ Random Forest classifiers [3] (100 estimators) to identify discriminative features:

Global Importance

Measures overall feature contribution across all classes:

$$\text{Importance}_{\text{global}}(f_i) = \frac{1}{T} \sum_{t=1}^T \sum_{n \in \text{splits}_t(f_i)} p_n \cdot \Delta I_n \quad (3.7)$$

Class-Specific Importance

Uses one-vs-rest classification to identify class-specialized features:

$$\text{Importance}_{\text{class}}(f_i, c) = \text{RF importance of } f_i \text{ for class } c \text{ vs rest} \quad (3.8)$$

3.5.2 Feature Manipulation Protocol

To understand feature semantics, we systematically activate individual features:

$$\mathbf{f}'_{i,v} = v \cdot \mathbf{e}_i, \quad v \in \{0, 0.25, 0.5, 0.75, 1.0\} \quad (3.9)$$

We analyze two configurations:

- **Moderate capacity:** $m = 400$, $\lambda = 10$, $\eta = 1 \times 10^{-3}$
- **High capacity:** $m = 1600$, $\lambda = 10$, $\eta = 1 \times 10^{-3}$

3.5.3 Feature Interaction Analysis

We create 2D activation grids for feature pairs:

$$\mathbf{f}'_{i,j}(v_i, v_j) = v_i \cdot \mathbf{e}_i + v_j \cdot \mathbf{e}_j \quad (3.10)$$

This reveals whether features operate independently or exhibit compositional semantics.

Part III

Experiments

/4

Results

4.1 Introduction

This chapter presents a comprehensive empirical evaluation of Sparse Autoencoders (SAEs) and Sparse Transformers (STs), validating the theoretical predictions developed in Chapter 2 through the systematic methodology outlined in Chapter 3. Our results reveal a striking pattern: these architectures exhibit complementary geometric biases that manifest consistently across multiple experimental domains, from simple visual datasets to complex language model representations.

The theoretical analysis in Chapter 2 predicted that architectural differences between SAEs and STs would lead to fundamentally different solutions to the sparse representation problem. Specifically, we hypothesized that SAEs' unbounded ReLU activations would enable superior Euclidean separation, while STs' normalized attention mechanisms would excel at angular discrimination. Our empirical findings not only confirm these predictions but reveal the magnitude and consistency of these differences across varying levels of complexity.

We structure our empirical investigation in three parts. First, Section 4.2 examines performance on toy datasets (MNIST and Fashion-MNIST), where controlled conditions allow us to isolate architectural effects and observe how they manifest across different hyperparameter settings. These experiments reveal extreme performance gaps, SAEs achieve 100-1000 $\times$ better Euclidean

separation while STs demonstrate $10\times$ superior cosine distances, validating our theoretical framework while uncovering unexpected dataset-dependent patterns.

Second, Section 4.3 provides detailed feature-level analysis, moving beyond aggregate metrics to understand how individual features contribute to class discrimination. This microscopic view reveals profound differences in how the architectures organize their learned representations: SAEs develop shared features that capture cross-class patterns, while STs learn highly specialized, class-specific features. These findings illuminate the mechanisms underlying the geometric biases observed in aggregate metrics.

Finally, Section 4.4 scales our analysis to real-world applications by examining GPT-Neo 1.3B representations. Here, we observe a notable convergence in performance gaps, suggesting that the extreme separation capabilities observed in controlled settings are moderated by the complexity of language model representations.

4.2 Comparative Hyperparameter Analysis

This section systematically examines how key hyperparameters influence the representation quality of both architectures. By varying feature dimensions, L1 regularization strength, and learning rates, we uncover the sensitivity profiles of each architecture and identify optimal configurations for different geometric objectives.

The theoretical framework from Chapter 2 suggests that SAEs' unbounded activations should enable them to scale Euclidean distances with increasing feature dimensions, while STs' normalization constraint should limit their Euclidean separation but enhance angular discrimination. Our experiments not only confirm these predictions but reveal nuanced patterns: optimal feature dimensions, differential responses to regularization, and striking differences in learning rate sensitivity that reflect the fundamental optimization landscapes of each architecture.

Feature Dimension Effects

The relationship between feature dimension and model performance reveals striking architectural differences and dataset-specific patterns.

Cosine Distance Performance

Architectural Hierarchy ST models consistently outperform SAE models by approximately one order of magnitude in cosine distance across both datasets, indicating superior angular class separation.

MNIST Observations As illustrated in Figure 4.1, ST models exhibit an optimization curve peaking at $f = 400$ dimensions, achieving cosine distances near 0.8. Performance degrades beyond this point, suggesting over-parameterization for MNIST's simple patterns. SAE models show volatile behavior with a notable performance dip at $f = 200$.

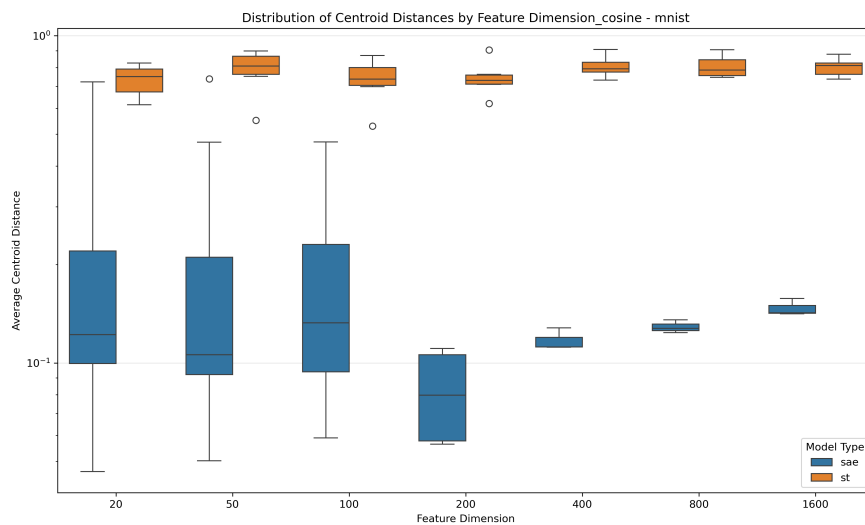


Figure 4.1: Effect of varying feature dimension on model performance measured by cosine distance for MNIST dataset. Boxes are slightly shifted for visibility but correspond to their nearest feature dimension.

Fashion-MNIST Observations The behavior on Fashion-MNIST, shown in Figure 4.2, presents markedly different characteristics. ST models maintain stable performance (0.7–0.8) across all dimensions without the peak-and-degradation pattern. SAE models show steady improvement without the volatility seen on MNIST, indicating that Fashion-MNIST’s complexity creates a more regularized optimization landscape.

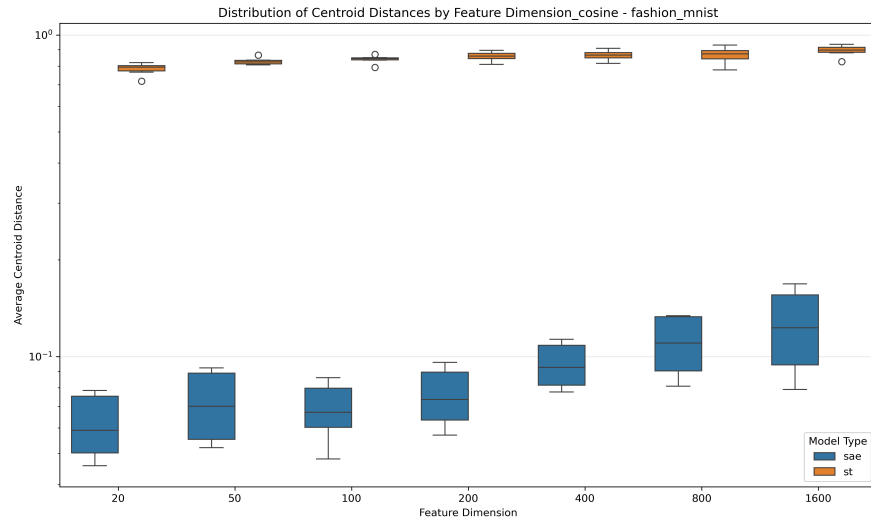


Figure 4.2: Effect of varying feature dimension on model performance measured by cosine distance for Fashion-MNIST dataset. Boxes are slightly shifted for visibility but correspond to their nearest feature dimension.

Euclidean Distance Performance The Euclidean distance analysis reveals the strengths of the SAE. SAE models now achieve centroid distances 2–3 orders of magnitude greater than ST variants, demonstrating fundamentally different geometric separation properties.

MNIST Analysis The SAE models in Figure 4.3 exhibit the same optimization trajectory previously seen in ST cosine performance: steady improvement to peak effectiveness around $f = 400$, followed by plateau and gradual degradation. This suggests consistent optimal feature utilization patterns across architectures.

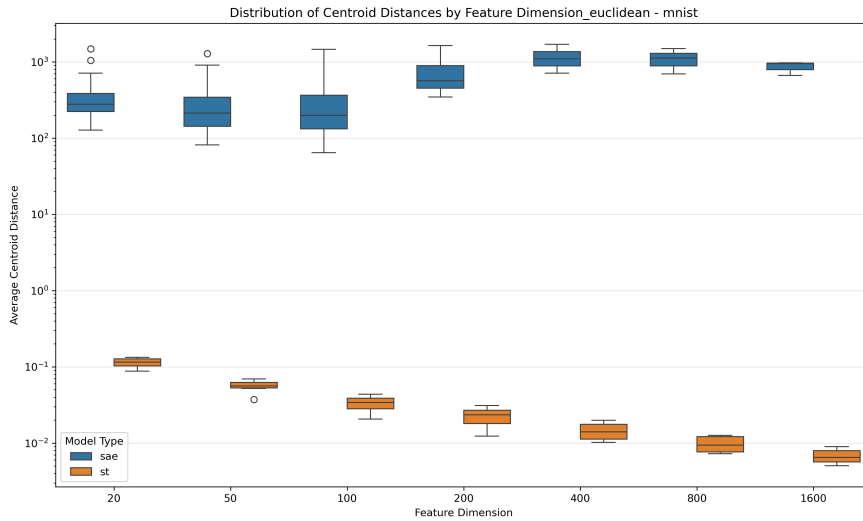


Figure 4.3: Effect of varying feature dimension on model performance measured by Euclidean distance for MNIST dataset. The logarithmic y-axis highlights the dramatic performance difference, with SAE models achieving centroid distances 100–1000 times larger than ST models.

Fashion-MNIST Analysis As demonstrated in Figure 4.4, SAE models show remarkable stability, maintaining centroid distances above 10^3 throughout the entire dimension range. This contrasts with their dimensional volatility in cosine metrics, indicating more regularized optimization for absolute geometric separation.

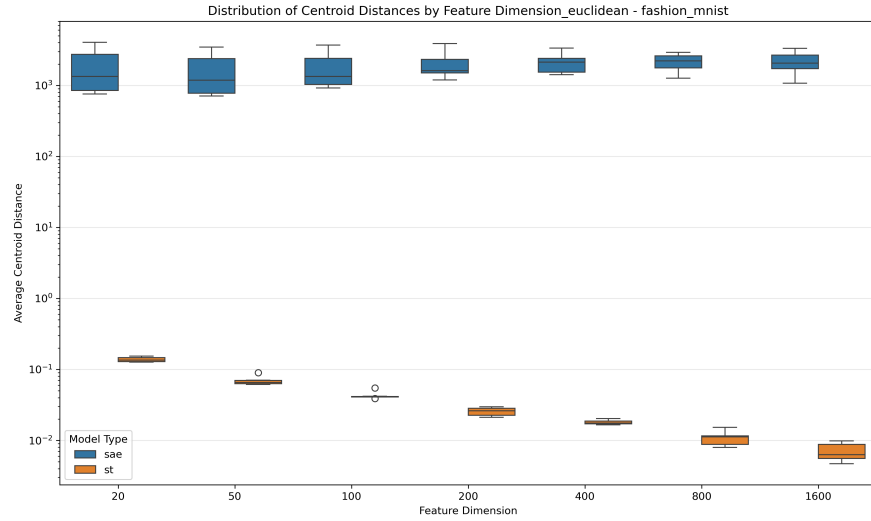


Figure 4.4: Effect of varying feature dimension on model performance measured by Euclidean distance for Fashion-MNIST dataset. The logarithmic y-axis reveals SAE models achieving 100–1000 $\times$ greater separation than ST models.

L1 Regularization Impact

L1 regularization reveals fundamentally different dataset sensitivities, with MNIST showing much stronger responsiveness to regularization strength compared to Fashion-MNIST.

Cosine Regularization Response

MNIST The left panel of Figure 4.5 demonstrates a clear positive correlation between L1 regularization strength (λ) and model performance. ST-Softmax exhibits the strongest response, achieving approximately 40% improvement when moving from $\lambda = 2.5$ to $\lambda = 10$.

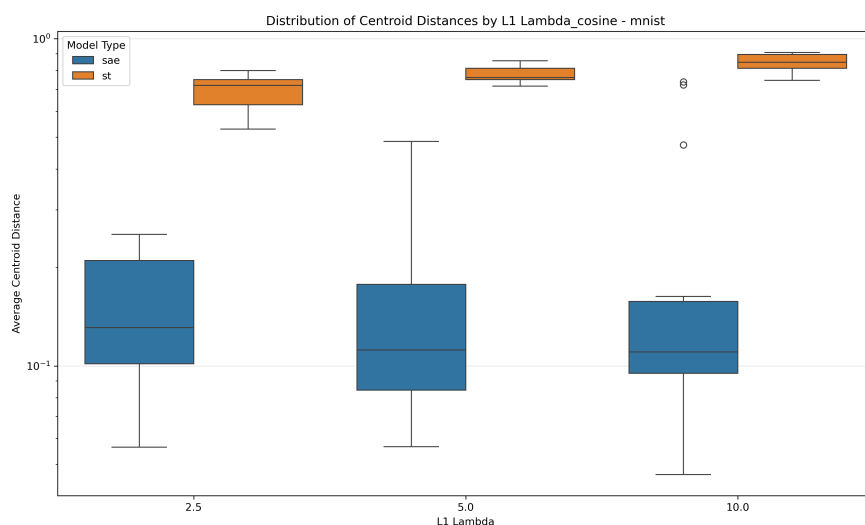


Figure 4.5: Impact of different L1 regularization strengths on model performance using cosine distance metric. MNIST dataset.

Fashion-MNIST Figure 4.6 reveals only modest increases in cosine centroid distance, with the same regularization range yielding only 15–20% improvement for ST-Softmax models.

Euclidean Distance Consistency For Euclidean distance metrics shown in Figure 4.7 and 4.8, SAE models exhibit consistent behavior across both datasets, showing similar monotonic increases in centroid distance with increasing λ values.

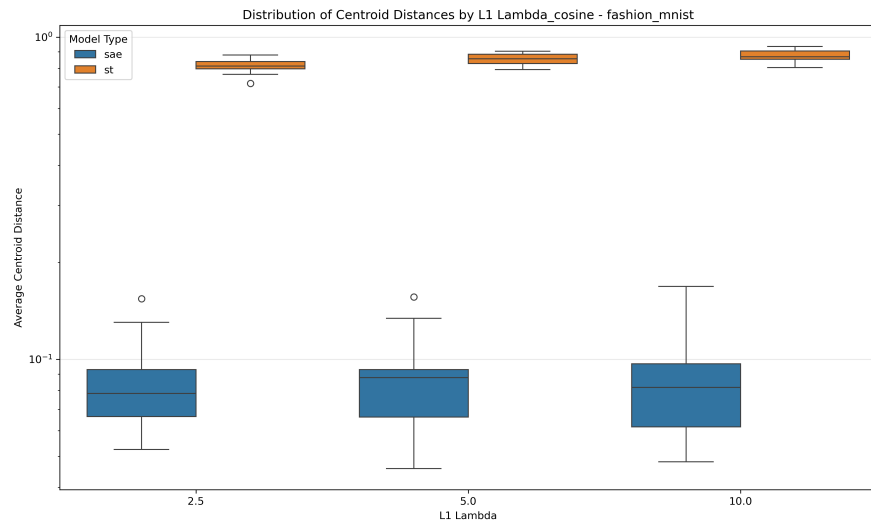


Figure 4.6: Impact of different L1 regularization strengths on model performance using cosine distance metric. Fashion-MNIST dataset.

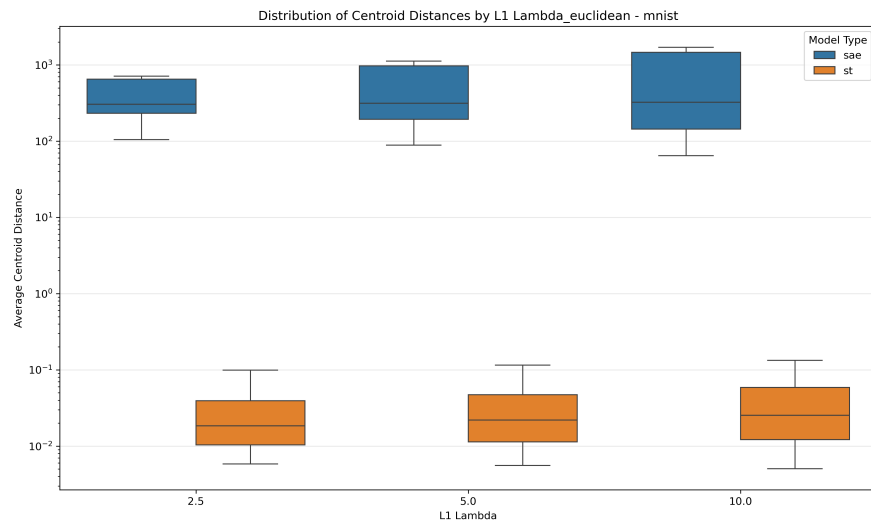


Figure 4.7: Impact of different L1 regularization strengths on model performance using Euclidean distance metric. MNIST dataset.

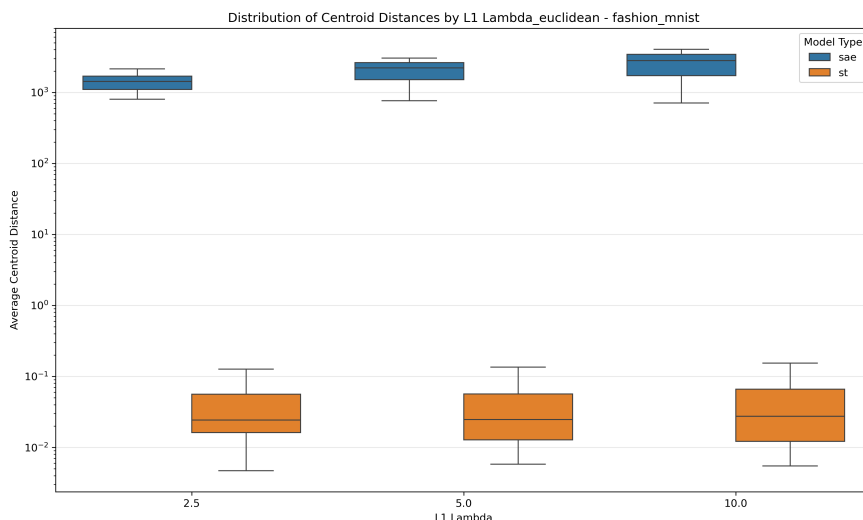


Figure 4.8: Impact of different L1 regularization strengths on model performance using Euclidean distance metric. Fashion-MNIST dataset.

Learning Rate Sensitivity

Learning rate variations reveal the most pronounced architectural differences between SAE and ST models, with these differences manifesting consistently across both datasets.

Architectural Response Patterns Both datasets demonstrate fundamental inverse relationships between SAE and ST models regarding learning rate optimization, as shown in Figures 4.9 and 4.10.

SAE Models Consistently improve with higher learning rates across both datasets. MNIST exhibits dramatic 4-fold improvement when increasing learning rate from $5 \cdot 10^{-5}$ to $1 \cdot 10^{-3}$, while Fashion-MNIST shows more gradual improvement with approximately half the relative gain.

ST Models Maintain remarkably consistent performance across all learning rates, with ST-Softmax showing stable behavior. This stability appears to be an inherent architectural characteristic.

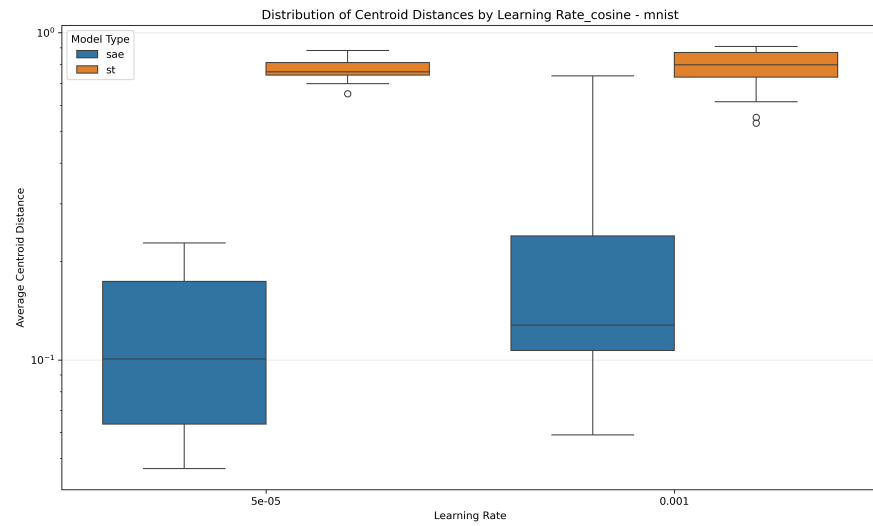


Figure 4.9: Effect of varying learning rates on model performance measured by cosine distance. MNIST dataset.

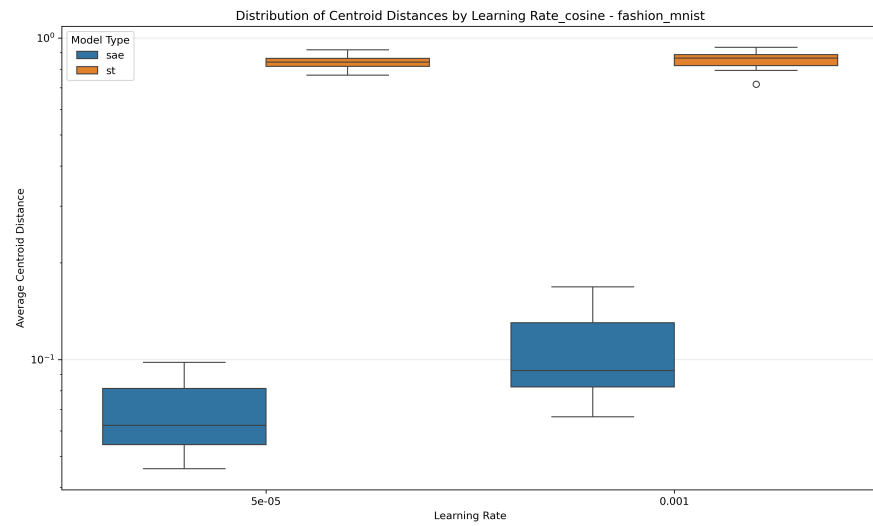


Figure 4.10: Effect of varying learning rates on model performance measured by cosine distance. Fashion-MNIST dataset.

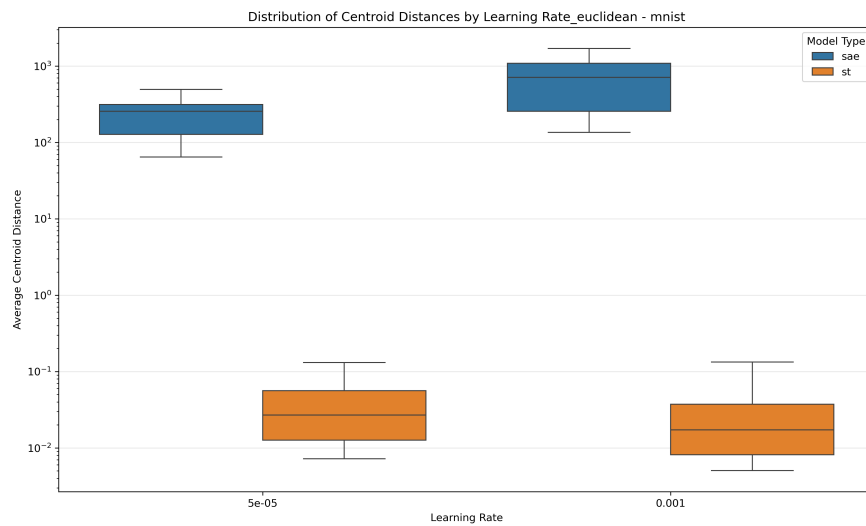


Figure 4.11: Effect of varying learning rates on model performance measured by Euclidean distance. MNIST dataset.

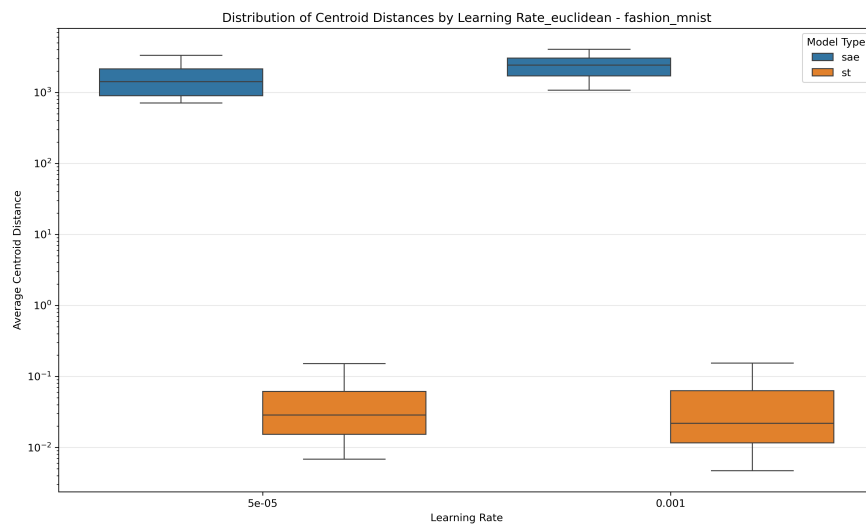


Figure 4.12: Effect of varying learning rates on model performance measured by Euclidean distance. Fashion-MNIST dataset.

4.3 Feature-Level Interpretability Analysis

While aggregate metrics reveal architectural biases in representation geometry, understanding how individual features contribute to class discrimination provides deeper insights into the mechanisms underlying these differences. This section employs the feature analysis methodology from Chapter 3 to examine the learned representations at a microscopic level.

The theoretical distinction between SAEs' independent feature activation and STs' competitive dynamics predicts fundamentally different feature specialization patterns. SAEs' ability to activate multiple features simultaneously should enable shared representations across classes, while STs' mutual exclusivity through softmax competition should drive features toward class-specific roles.

We analyze two model configurations for each architecture: moderate capacity ($m = 400$) and high capacity ($m = 1600$), both trained with strong regularization ($\lambda = 10$) to ensure sparse, interpretable features. Through Random Forest-based importance analysis, systematic feature manipulation, and interaction visualization, we reveal how these architectural differences manifest in practice.

4.3.1 Feature Variation Effects on MNIST

SAE Feature Analysis

400-Dimensional Features The feature importance distribution for the 400-dimensional SAE model follows a heavy-tailed pattern. The most important feature captures approximately 22% of the total importance, while the remaining features show decreasing importance values. Figure 4.13 displays this distribution.

The class-specific feature importance patterns are visualized in Figure 4.14. Feature F142 appears in the top 5 for all 10 classes, ranking first for classes 0, 4, 6, 7, and 9. Feature F210 appears in 7 classes' top 5 features, consistently ranking second. Feature F326 ranks first for classes 3, 5, and 8, appearing in 8 total classes.

Single-class specialized features include F220 (class 1), F296 (class 2), F59 (class 6), F27 (class 3), F339 (class 3), F341 (class 2), F16 (class 7), F51 (class 9), and F95-F96 (class 5).

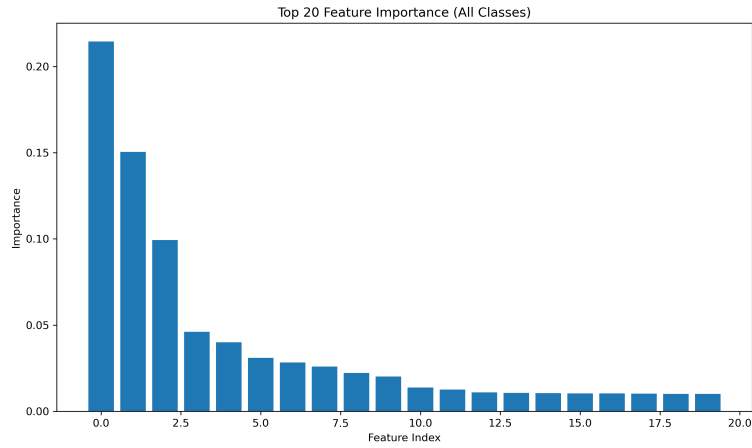


Figure 4.13: Feature importance distribution for SAE with $f = 400$ on MNIST.

Feature	0	1	2	3	4	5	6	7	8	9
F142	1	2	4	5	1	2	3	1	4	1
F210	2	-	2	-	2	-	2	2	2	2
F326	-	4	3	1	5	1	-	3	1	5
F59	-	-	-	-	-	-	1	-	-	-
F220	-	1	-	-	-	-	-	-	-	-
F296	-	-	1	-	-	-	-	-	-	-
F89	-	-	-	-	3	-	-	-	-	3
F300	-	-	-	-	-	3	-	-	3	-
F338	3	-	-	-	-	-	-	-	-	-
F48	4	-	-	-	-	-	-	-	-	-
F23	5	-	-	-	-	-	-	-	-	-
F55	-	3	-	-	-	-	-	-	-	-
F341	-	-	5	-	-	-	-	-	-	-
F27	-	-	-	2	-	-	-	-	-	-
F339	-	-	-	3	-	-	-	-	-	-

Figure 4.14: Feature importance heatmap for 400D SAE showing top 5 rankings per class. Numbers indicate rank (1-5), with darker shading for higher ranks. Dash indicates feature not in top 5. Shows 25 most important features across all classes.

1600-Dimensional Features The feature importance distribution for the 1600-dimensional SAE model shows a more distributed feature importance. The top feature captures 2% of the total importance, while having a less dramatic tailing-effect. The rest of the features also show greater importance in relation to the top feature. Figure 4.15 shows this distribution.

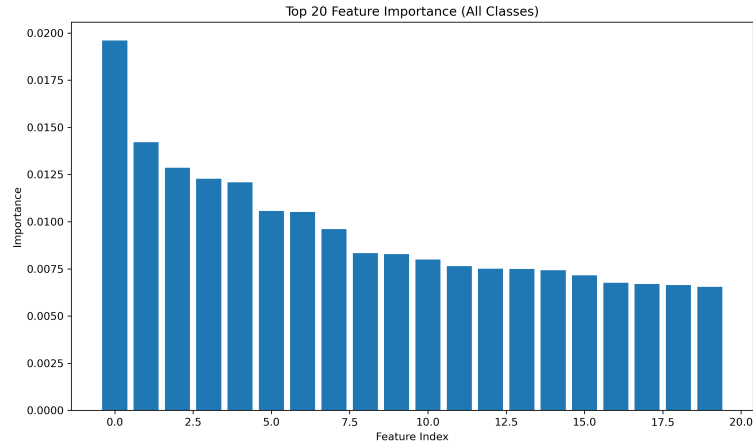


Figure 4.15: Feature importance distribution for SAE with $f = 1600$ on MNIST.

The class-specific analysis shows minimal feature sharing across classes. Only 2 features appear in multiple classes' top 5: F321 (classes 5 and 8) and F1041 (classes 7 and 9). Figure 4.16 displays the top features for each class.

Feature	0	1	2	3	4	5	6	7	8	9
F973	-	1	-	-	-	-	-	-	-	-
F1041	-	-	-	-	-	-	-	1	-	3
F519	1	-	-	-	-	-	-	-	-	-
F1539	-	-	-	-	1	-	-	-	-	-
F1296	-	-	-	1	-	-	-	-	-	-
F506	-	-	1	-	-	-	-	-	-	-
F1597	-	-	-	-	-	1	-	-	-	-
F1073	-	-	-	-	-	-	1	-	-	-
F384	-	-	-	-	-	-	-	-	1	-
F1475	-	-	-	-	-	-	-	-	-	1
F321	-	-	-	-	-	4	-	-	2	-
F819	-	-	2	-	-	-	-	-	-	-
F1113	2	-	-	-	-	-	-	-	-	-
F671	-	-	-	-	2	-	-	-	-	-
F67	-	-	-	2	-	-	-	-	-	-

Figure 4.16: Feature importance heatmap for 1600D SAE showing top-ranked features per class. Most features appear for single classes only. Numbers indicate rank (1-5), with darker shading for higher ranks.

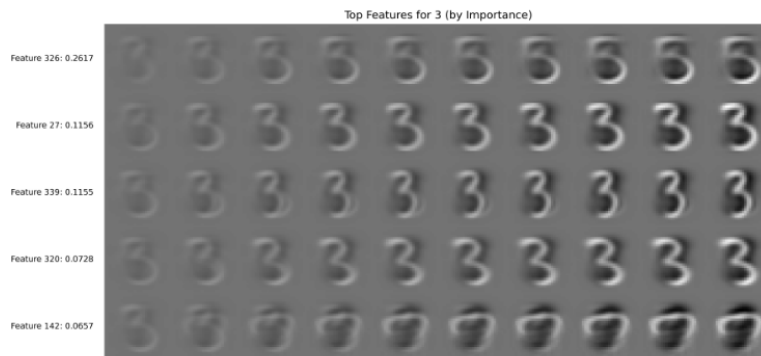


Figure 4.17: Top 5 most important features for the digit 3 with SAE 400D model.

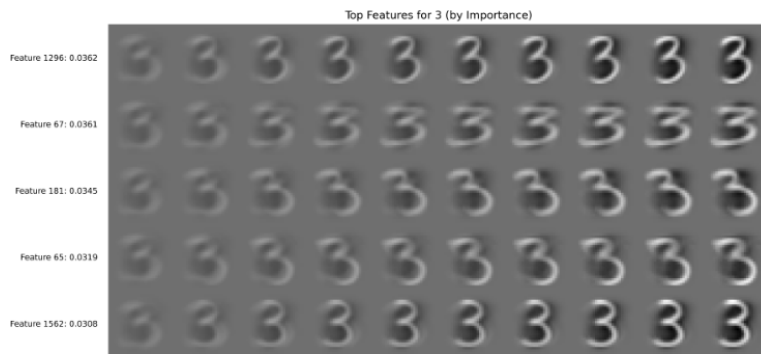


Figure 4.18: Top 5 most important features for the digit 3 with SAE 1600D model.

The class-specific importance values vary significantly:

- Class 1: F973 (20.0%) - highest single feature importance
- Class 7: F1041 (10.1%)
- Classes 0, 4, 6, 8, 9: Top features range from 6.8% to 8.9%
- Classes 2, 3, 5: Top features range from 3.4% to 6.0%

Polysemanticity and Feature Capacity

Figure 4.19 and 4.20 reveal a fundamental transition in feature representation as capacity increases. The 400-dimensional model exhibits signs of polysemanticity, with individual features encoding multiple digit concepts. This aligns with the superposition hypothesis (Equation 2.3), where limited capacity forces the network to compress multiple semantic concepts into shared directions.

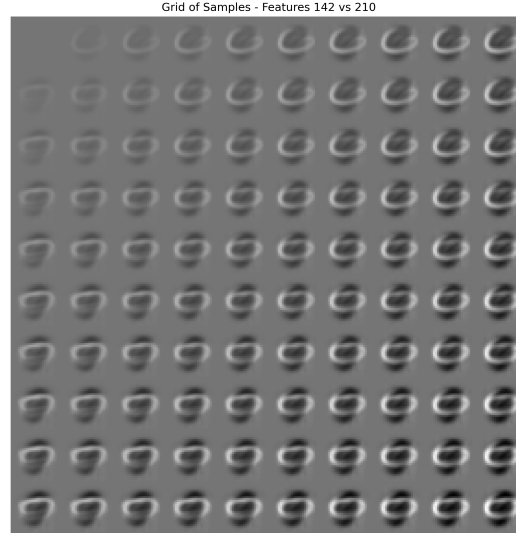


Figure 4.19: Feature interaction grid for SAE model using 400-dimensional latent space (features 142 and 210).

Feature 142, appearing as the most important feature across multiple classes, exemplifies this compression, it captures general curved structures common to digits 0, 3, 5, 6, 8, and 9.

In contrast, the 1600-dimensional model ($m > n = 784$) achieves seemingly monosemantic representations, with each feature cleanly reconstructing a single digit class. This transition occurs precisely at the theoretical threshold where the model has sufficient capacity to assign unique basis vectors to each concept without requiring superposition. The feature combinations in the 1600D model produce simple linear superpositions rather than the complex nonlinear interactions observed in the 400D model, confirming that increased capacity enables disentangled representations.

The class-specific analysis for digit 3 (Figures 4.17 and 4.18) further illustrates this distinction. In the 400D model, only 3 of the 5 most important features can individually reconstruct digit 3, with the top feature (F142) resembling a 5, indicating shared representation across multiple classes. The 1600D model shows all 5 top features capable of independently reconstructing digit 3, demonstrating successful feature disentanglement. This empirical validation of the polysemanticity-to-monosemanticity transition provides direct evidence for the theoretical prediction that overcomplete representations ($m \gg n$) enable interpretable, class-specific features.

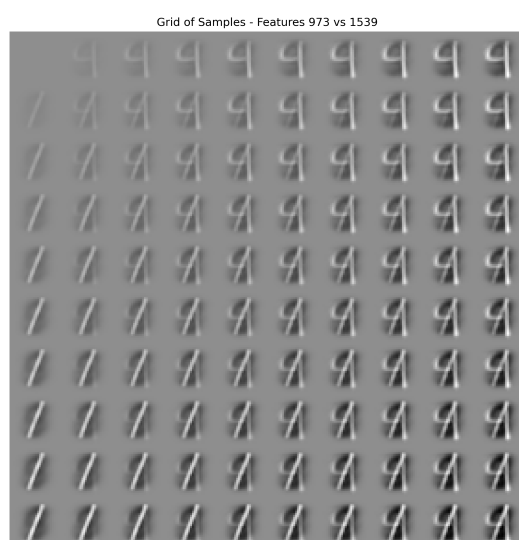


Figure 4.20: Feature interaction grid for SAE model using 1600-dimensional latent space (features 973 and 1539).

ST Feature Analysis

400-Dimensional Features The feature importance distribution for the 400-dimensional ST model shows a more gradual decay compared to SAE. The top feature captures 2.8% of total importance, indicating more distributed representations. Figure 4.21 displays this distribution.

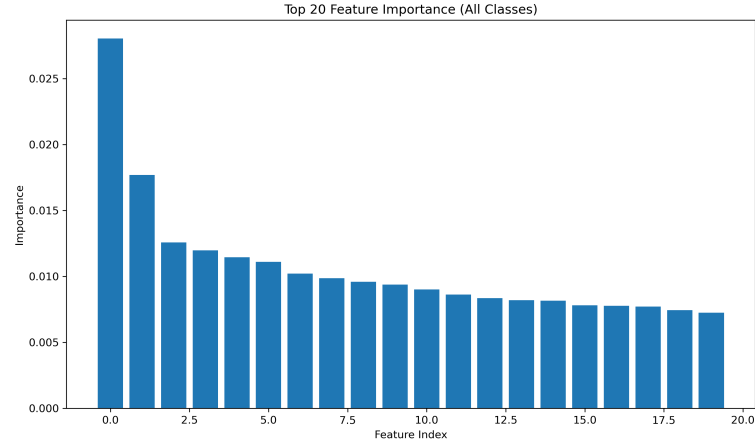


Figure 4.21: Feature importance distribution for ST with $f = 400$ on MNIST.

The class-specific feature importance patterns reveal complete specialization, no features appear in the top 5 for multiple classes. Figure 4.22 visualizes this extreme class-specific encoding.

Feature	0	1	2	3	4	5	6	7	8	9
F378	-	1	-	-	-	-	-	-	-	-
F389	-	-	-	-	-	1	-	-	-	-
F224	-	-	-	1	-	-	-	-	-	-
F325	-	-	1	-	-	-	-	-	-	-
F165	1	-	-	-	-	-	-	-	-	-
F89	-	-	-	-	-	-	1	-	-	-
F352	-	-	-	-	-	-	-	1	-	-
F249	-	-	-	-	1	-	-	-	-	-
F187	-	-	-	1	-	-	-	-	-	-
F392	-	-	-	-	-	-	-	-	-	1
F388	-	2	-	-	-	-	-	-	-	-
F377	-	-	-	-	2	-	-	-	-	-
F135	-	-	2	-	-	-	-	-	-	-
F252	2	-	-	-	-	-	-	-	-	-
F19	-	-	-	-	-	-	2	-	-	-

Figure 4.22: Feature importance heatmap for 400D ST showing top 5 rankings per class. Complete specialization with no feature sharing across classes. Numbers indicate rank, with darker shading for higher ranks.

The importance values vary across classes:

- Class 1: F378 (9.0%) - highest single feature importance
- Class 4: F224 (5.8%)
- Class 6: F389 (5.5%)
- Class 2: F325 (5.4%)
- Classes 0, 7, 8: Top features range from 4.6% to 4.7%
- Classes 3, 5: Top features around 3.8-4.0%
- Class 9: F392 (3.1%) - lowest top feature importance

1600-Dimensional Features The feature importance distribution for the 1600-dimensional ST model shows further flattening. The top feature captures only 1.9% of total importance. Figure 4.23 shows this distribution.

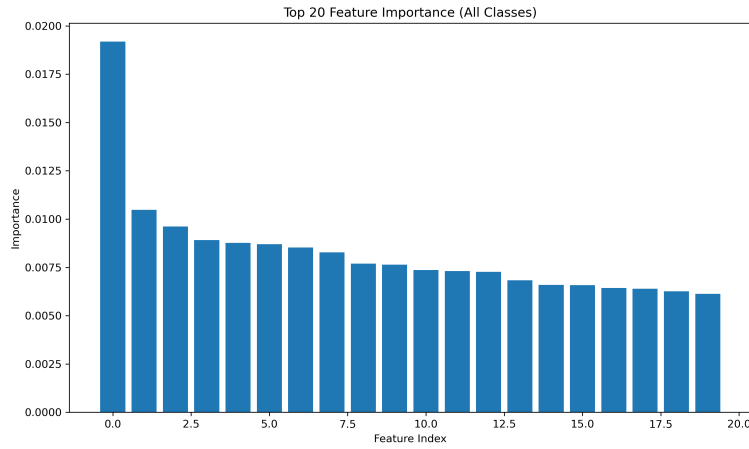


Figure 4.23: Feature importance distribution for ST with $f = 1600$ on MNIST.

The class-specific analysis maintains complete feature specialization, no features appear in the top 5 for multiple classes. Figure 4.24 displays the top features for each class.

Feature	0	1	2	3	4	5	6	7	8	9
F777	-	-	-	-	-	-	-	1	-	-
F857	-	1	-	-	-	-	-	-	-	-
F754	-	-	1	-	-	-	-	-	-	-
F1296	1	-	-	-	-	-	-	-	-	-
F1297	-	-	-	1	-	-	-	-	-	-
F319	-	-	-	-	1	-	-	-	-	-
F813	-	-	-	-	-	1	-	-	-	-
F1223	-	-	-	-	-	-	1	-	-	-
F454	-	-	-	-	-	-	-	-	1	-
F447	-	-	-	-	-	-	-	-	-	1
F529	-	-	-	-	-	-	-	2	-	-
F57	-	2	-	-	-	-	-	-	-	-
F606	-	-	2	-	-	-	-	-	-	-
F161	2	-	-	-	-	-	-	-	-	-
F387	-	-	-	2	-	-	-	-	-	-

Figure 4.24: Feature importance heatmap for 1600D ST showing top-ranked features per class. Complete specialization maintained with no feature sharing. Numbers indicate rank, with darker shading for higher ranks.

The importance values show more uniform distribution across classes:

- Class 7: F777 (5.1%) - highest single feature importance
- Class 1: F857 (5.0%)
- Class 2: F754 (3.9%)
- Class 0: F1296 (3.8%)
- Class 5: F813 (3.5%)
- Classes 4, 6, 9: Top features range from 3.0% to 3.2%
- Class 8: F454 (3.0%)
- Class 3: F1297 (2.6%) - lowest top feature importance

Competitive Dynamics and Noise-Based Encoding

Figure 4.27 and 4.28 reveal an unexpected mechanism by which STs circumvent their architectural constraints. The prevalence of noise-like patterns in important features suggests that STs encode discriminative information through distributed activation patterns rather than clean geometric templates. This phenomenon emerges from the fundamental tension between the softmax normalization constraint (Equation 2.12) and the need for large Euclidean distances

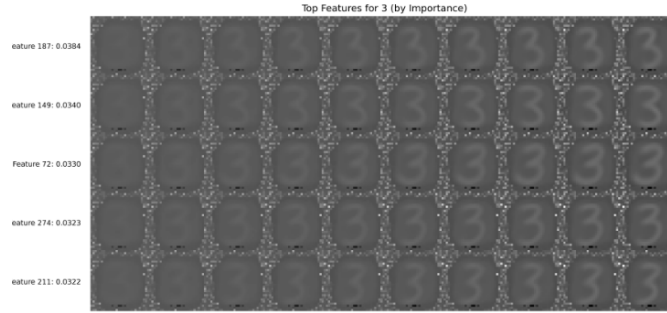


Figure 4.25: Top 5 most important features for the digit 3 with ST 400D model.

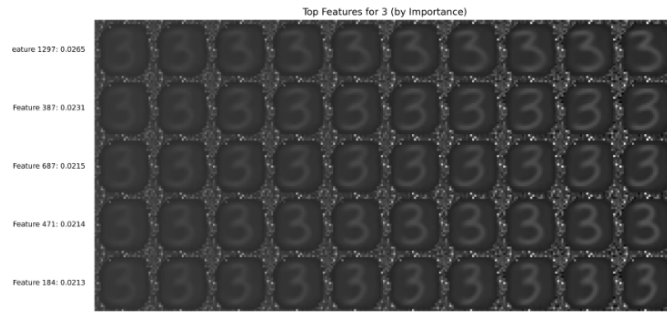


Figure 4.26: Top 5 most important features for the digit 3 with ST 1600D model.

between class representations.

The softmax mechanism forces all features to compete for a fixed probability budget ($\sum_i f_i = 1$), preventing any single feature from achieving the large activation magnitudes that SAEs use for Euclidean separation. In response, STs appear to encode class information in the relative activation patterns across many features, manifesting as apparent "noise" when individual features are visualized. This distributed encoding strategy allows STs to maintain discriminative power despite bounded activations, explaining their strong angular separation performance while struggling with Euclidean metrics.

Remarkably, the 400D ST model achieves better monosemantic representations than the corresponding SAE model, with cleaner digit-specific features despite being under-parameterized. This suggests that the competitive dynamics induced by softmax (Equation 2.13) provide an implicit regularization effect, forcing features to specialize more aggressively to win the attention competition. The winner-take-all dynamics appear particularly effective when capacity is limited, preventing the feature sharing that leads to polysemanticity in SAEs.

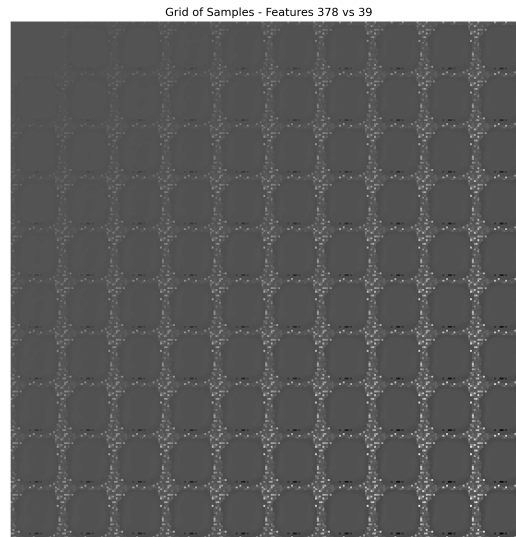


Figure 4.27: Feature interaction grid for ST model using 400-dimensional latent space (features 378 and 39).

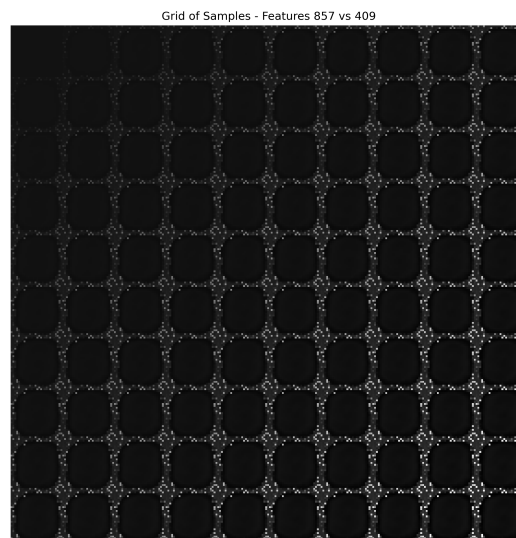


Figure 4.28: Feature interaction grid for ST model using 1600-dimensional latent space (features 857 and 409).

4.3.2 Feature Variation Effects on Fashion-MNIST

SAE Feature Analysis

400-Dimensional Features The feature importance distribution for the 400-dimensional SAE model exhibits a more gradual decay compared to MNIST. The top feature captures 1.7% of total importance, substantially lower than MNIST’s concentrated distribution. Figure 4.29 displays this distribution. The

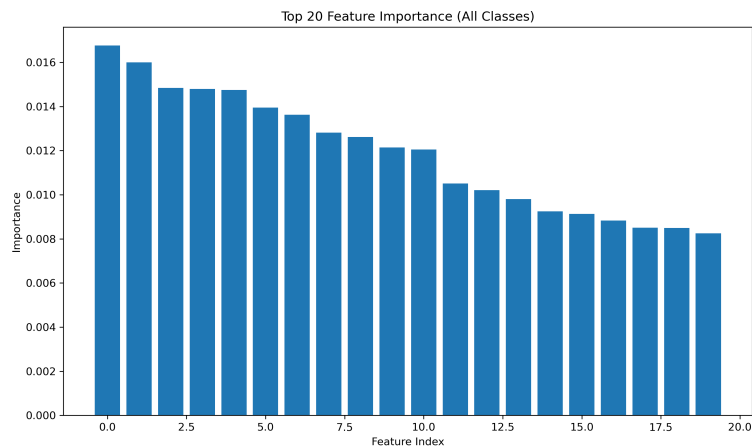


Figure 4.29: Feature importance distribution for SAE with f=400 on Fashion-MNIST.

class-specific feature importance patterns reveal complete specialization—no features appear in the top 5 for multiple classes. Figure 4.30 visualizes this extreme class-specific encoding.

Feature	0	1	2	3	4	5	6	7	8	9
F298	1	-	-	-	-	-	-	-	-	-
F205	-	1	-	-	-	-	-	-	-	-
F377	-	-	1	-	-	-	-	-	-	-
F266	-	-	-	1	-	-	-	-	-	-
F244	-	-	-	-	1	-	-	-	-	-
F49	-	-	-	-	-	1	-	-	-	-
F58	-	-	-	-	-	-	1	-	-	-
F177	-	-	-	-	-	-	-	1	-	-
F179	-	-	-	-	-	-	-	-	1	-
F176	-	-	-	-	-	-	-	-	-	1
F154	2	-	-	-	-	-	-	-	-	-
F39	-	2	-	-	-	-	-	-	-	-
F142	-	-	3	-	-	-	-	-	-	-
F152	-	-	-	2	-	-	-	-	-	-
F360	-	-	-	-	2	-	-	-	-	-

Figure 4.30: Feature importance heatmap for 400D SAE showing top 5 rankings per class. Complete specialization with no feature sharing across classes. Numbers indicate rank, with darker shading for higher ranks.

The importance values vary dramatically across clothing categories:

- Class 7 (Sneaker): F177 (6.8%) - highest single feature importance
- Class 9 (Ankle boot): F176 (6.1%)
- Class 1 (Trouser): F205 (6.0%)
- Class 5 (Sandal): F49 (4.9%)
- Classes 0, 8: Top features around 4.4-4.5%
- Classes 2, 3: Top features around 3.5-3.7%
- Classes 4, 6: Top features only 2.8-3.4% - lowest importance values

1600-Dimensional Features The feature importance distribution for the 1600-dimensional SAE model shows further flattening. The top feature captures only 1.2% of total importance. Figure 4.31 shows this distribution. The class-

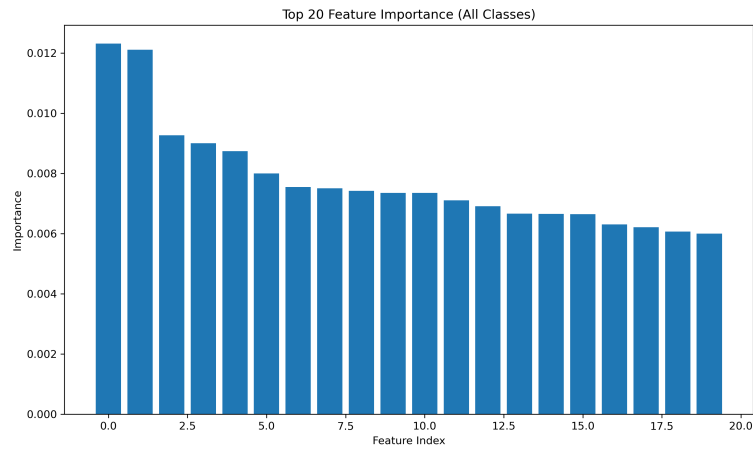


Figure 4.31: Feature importance distribution for SAE with $f=1600$ on Fashion-MNIST.

specific analysis shows limited feature sharing. Only 3 features appear in multiple classes' top 5: F58 (classes 4 and 6), F1316 (classes 4 and 6), and F94 (classes 5 and 7). Figure 4.32 displays the top features for each class.

Feature	0	1	2	3	4	5	6	7	8	9
F992	-	-	-	-	-	-	-	1	-	-
F1019	1	-	-	-	-	-	-	-	-	-
F1428	-	1	-	-	-	-	-	-	-	-
F813	-	-	1	-	-	-	-	-	-	-
F1178	-	-	-	1	-	-	-	-	-	-
F774	-	-	-	-	1	-	-	-	-	-
F82	-	-	-	-	-	1	-	-	-	-
F493	-	-	-	-	-	-	1	-	-	-
F1328	-	-	-	-	-	-	-	-	1	-
F799	-	-	-	-	-	-	-	-	-	1
F58	-	-	-	-	2	-	3	-	-	-
F1316	-	-	-	-	5	-	5	-	-	-
F94	-	-	-	-	-	4	-	2	-	-
F1473	2	-	-	-	-	-	-	-	-	-
F669	-	2	-	-	-	-	-	-	-	-

Figure 4.32: Feature importance heatmap for 1600D SAE showing top-ranked features per class. Limited sharing emerges: F58 and F1316 shared by classes 4 and 6; F94 shared by classes 5 and 7. Numbers indicate rank, with darker shading for higher ranks.

The importance values show more uniform distribution across classes:

- Class 7: F992 (4.6%) - highest single feature importance
- Class 0: F1019 (4.5%)
- Classes 1, 2: Top features range from 3.0% to 4.2%
- Classes 3, 4, 5, 8, 9: Top features range from 2.4% to 2.9%
- Class 6: F493 (1.8%) - lowest top feature importance

Figures 4.33 and 4.34 displays 2D feature interaction grids. The 400-dimensional model grid shows variations of features 191 and 56. The 1600-dimensional model grid shows variations of features 1099 and 1253.

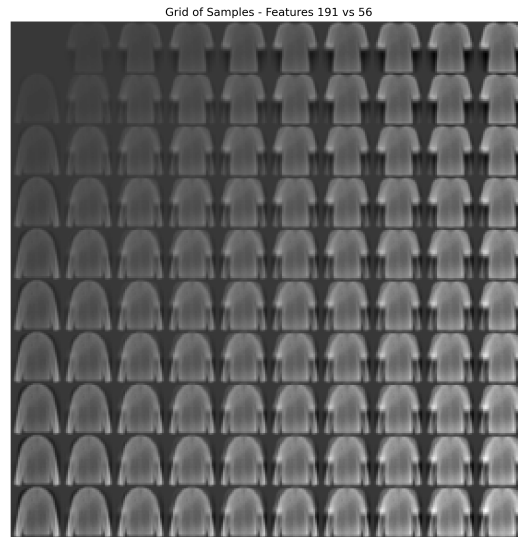


Figure 4.33: Feature interaction grid for SAE model on Fashion-MNIST using 400-dimensional latent space (features 191 and 56).

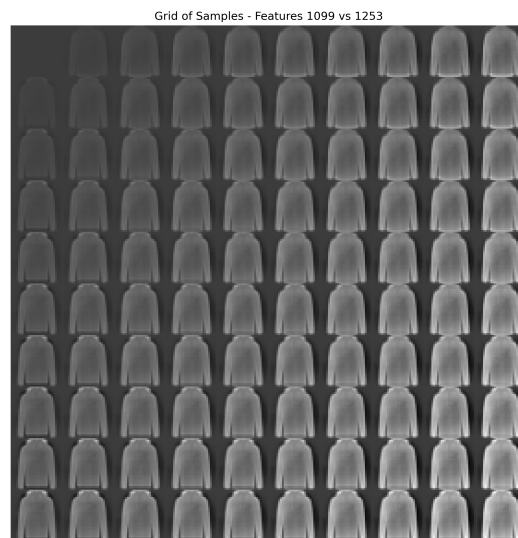


Figure 4.34: Feature interaction grid for SAE model on Fashion-MNIST using 1600-dimensional latent space (features 1099 and 1253).

ST Feature Analysis

400-Dimensional Features The feature importance distribution for the 400-dimensional ST model shows moderate concentration. The top feature captures 1.5% of total importance, indicating more distributed representations than the corresponding SAE model. Figure 4.35 displays this distribution.

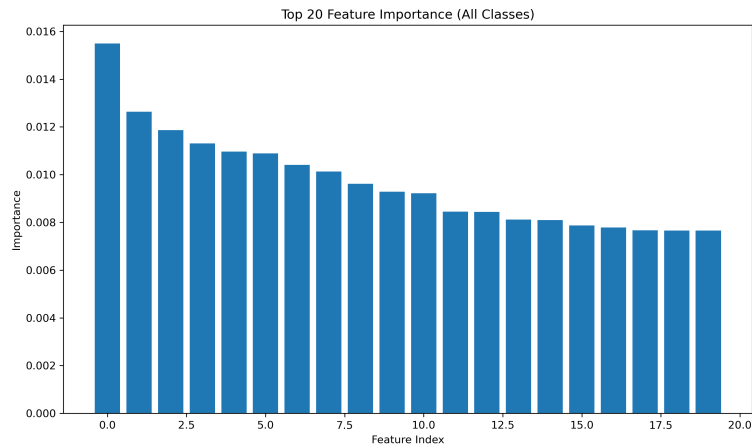


Figure 4.35: Feature importance distribution for ST with $f = 400$ on Fashion-MNIST.

The class-specific feature importance patterns reveal near-complete specialization, with only one feature (F93) appearing in multiple classes' top 5. Figure 4.36 visualizes this pattern.

Feature	0	1	2	3	4	5	6	7	8	9
F172	-	-	-	-	-	-	-	1	-	-
F289	1	-	-	-	-	-	-	-	-	-
F59	-	1	-	-	-	-	-	-	-	-
F333	-	-	1	-	-	-	-	-	-	-
F88	-	-	-	1	-	-	-	-	-	-
F352	-	-	-	-	1	-	-	-	-	-
F227	-	-	-	-	-	1	-	-	-	-
F101	-	-	-	-	-	-	1	-	-	-
F110	-	-	-	-	-	-	-	1	-	-
F360	-	-	-	-	-	-	-	-	-	1
F8	2	-	-	-	-	-	-	-	-	-
F4	-	2	-	-	-	-	-	-	-	-
F209	-	-	2	-	-	-	-	-	-	-
F244	-	-	-	2	-	-	-	-	-	-
F368	-	-	-	-	2	-	-	-	-	-

Figure 4.36: Feature importance heatmap for 400D ST showing top 5 rankings per class. Near-complete specialization with only F93 shared between classes 4 and 6. Numbers indicate rank, with darker shading for higher ranks.

The importance values vary significantly across clothing categories:

- Class 7 (Sneaker): F172 (7.2%) - highest single feature importance
- Class 1 (Trouser): F59 (6.8%)
- Class 0 (T-shirt/top): F289 (6.4%)
- Class 3 (Dress): F88 (6.2%)
- Class 2 (Pullover): F333 (6.0%)
- Class 8 (Bag): F110 (4.7%)
- Class 5 (Sandal): F227 (4.6%)
- Class 9 (Ankle boot): F360 (4.2%)
- Class 4 (Coat): F352 (3.6%)
- Class 6 (Shirt): F101 (2.4%) - lowest top feature importance

1600-Dimensional Features The feature importance distribution for the 1600-dimensional ST model shows further flattening. The top feature captures 1.7% of total importance. Figure 4.37 shows this distribution.

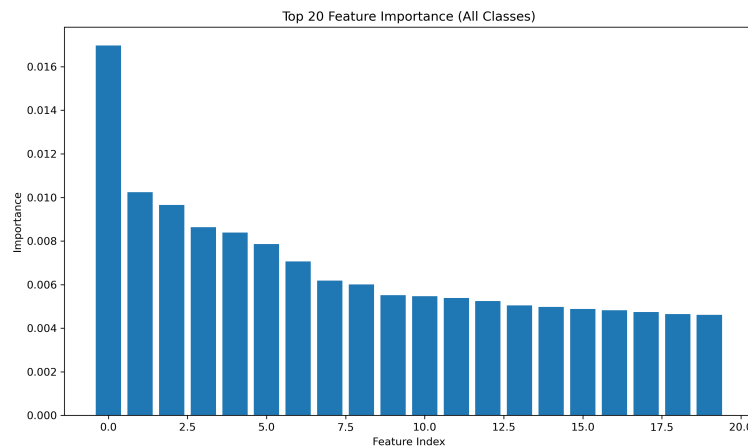


Figure 4.37: Feature importance distribution for ST with $f = 1600$ on Fashion-MNIST.

The class-specific analysis shows complete feature specialization—no features appear in the top 5 for multiple classes. Figure 4.38 displays the top features for each class.

Feature	0	1	2	3	4	5	6	7	8	9
F1596	-	-	-	-	-	-	-	1	-	-
F613	-	-	-	1	-	-	-	-	-	-
F1241	-	1	-	-	-	-	-	-	-	-
F542	-	2	-	-	-	-	-	-	-	-
F165	-	-	-	2	-	-	-	-	-	-
F963	-	-	-	-	-	-	-	-	-	1
F937	-	-	-	-	-	-	-	2	-	-
F170	-	3	-	-	-	-	-	-	-	-
F1208	-	-	-	-	-	-	-	3	-	-
F1303	1	-	-	-	-	-	-	-	-	-
F922	2	-	-	-	-	-	-	-	-	-
F33	3	-	-	-	-	-	-	-	-	-
F1348	-	-	1	-	-	-	-	-	-	-
F592	-	-	-	-	1	-	-	-	-	-
F431	-	-	-	-	-	1	-	-	-	-

Figure 4.38: Feature importance heatmap for 1600D ST showing top-ranked features per class. Complete specialization with no feature sharing. Numbers indicate rank, with darker shading for higher ranks.

The importance values show more variability across classes:

- Class 7: F1596 (7.6%) - highest single feature importance
- Class 3: F613 (3.9%)
- Class 1: F1241 (3.8%)
- Class 9: F963 (3.5%)
- Class 0: F1303 (3.1%)
- Class 8: F996 (2.8%)
- Class 2: F1348 (2.5%)
- Class 4: F592 (2.1%)
- Class 5: F431 (1.9%)
- Class 6: F589 (1.9%) - lowest top feature importance

Figures 4.39 and 4.40 displays 2D feature interaction grids. The 400-dimensional model grid shows variations of features 59 and 172. The 1600-dimensional model grid shows variations of features 1596 and 1453.

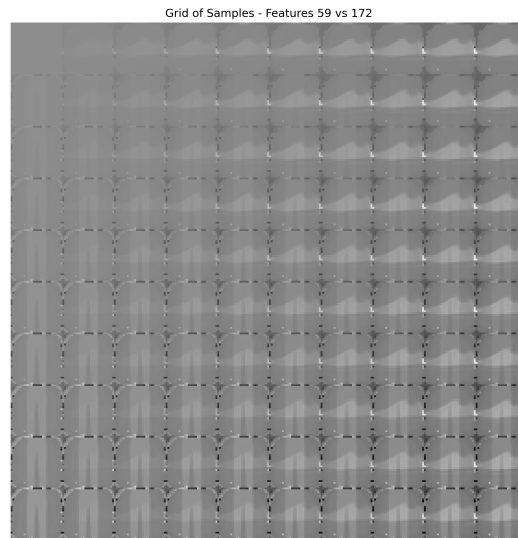


Figure 4.39: Feature interaction grid for ST model (400D). Features 59 and 172.

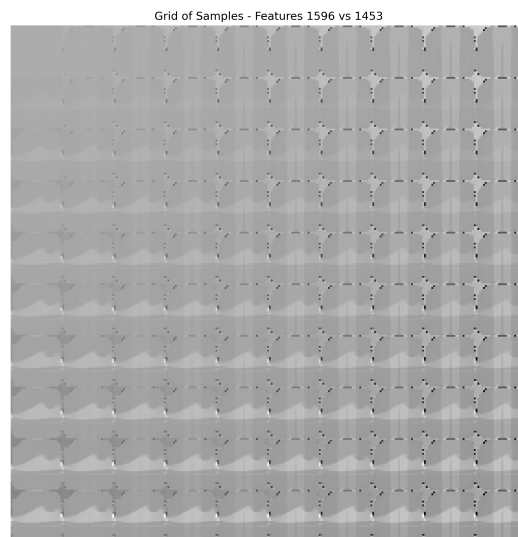


Figure 4.40: Feature interaction grid for ST model (1600D). Features 1596 and 1453.

4.3.3 Dataset Complexity and Feature Learning

The Fashion-MNIST results reveal how dataset complexity modulates architectural differences. Unlike MNIST's simple geometric patterns, Fashion-MNIST's textures and subtle class boundaries create a more challenging representation learning task that affects both architectures.

Convergence of Feature Specialization

Both architectures show increased feature specialization on Fashion-MNIST compared to MNIST. The 400D SAE exhibits complete class-specific specialization (Figure 4.30), abandoning the extensive feature sharing observed in MNIST. This suggests that the complexity of Fashion-MNIST's patterns prevents effective feature reuse, textures and clothing structures are sufficiently distinct that shared representations offer no advantage. The ST models maintain their complete specialization pattern.

Architectural Robustness to Complexity

Despite the increased challenge, both architectures successfully learn interpretable features for Fashion-MNIST, demonstrating the robustness of sparse representation learning. The SAE's transition from shared to specialized features shows adaptive capacity: the architecture naturally adjusts its feature allocation strategy based on dataset properties. The stability of ST's specialization pattern across datasets confirms that competitive dynamics consistently drive features toward class-specific roles, regardless of input complexity.

The feature interaction grids (Figures 4.33, 4.34, 4.39 and 4.40) reveal that both models capture meaningful clothing structures. SAE features show recognizable clothing items with smooth textures, while ST features maintain their low-contrast, slightly noisy reconstructions.

Most significantly, the performance metrics (Sections 4.29 through 4.37) show that both architectures achieve effective class separation despite Fashion-MNIST’s challenges. The top features consistently capture 2-7% of class-specific importance, indicating successful feature discovery. This convergence in feature quality, despite fundamental architectural differences, validates the theoretical framework: both unbounded sparse activations (SAE) and normalized competitive dynamics (ST) can learn interpretable representations when provided sufficient capacity and appropriate regularization.

4.4 Application to Large Language Model Representations

The ultimate test of our theoretical framework lies in its applicability to real-world scenarios. This section examines whether the architectural biases observed in controlled experiments persist when applied to high-dimensional representations from large language models. Using GPT-Neo 1.3B as a testbed, we analyze intermediate layer representations (layers 12-16) with dramatically increased scale: 2048-dimensional inputs and 8000-dimensional sparse features.

This scaling introduces several challenges that test the robustness of our findings. First, the complexity of language model representations may overwhelm the simple geometric biases observed in toy datasets. Second, the increased dimensionality may allow both architectures to achieve adequate separation regardless of their theoretical constraints. Finally, the hierarchical nature of transformer representations, where different layers capture different levels of abstraction, provides an additional dimension for comparing architectural behavior.

The results reveal both consistency and surprises: while the fundamental geometric biases persist, their magnitude diminishes substantially, suggesting that real-world applications may benefit from hybrid approaches that leverage the complementary strengths of both architectures.

4.4.1 Experimental Setup

To evaluate how our findings scale to real-world applications, we applied both SAE and ST architectures to intermediate representations from GPT-Neo 1.3B. We analyzed layers 12-16, training separate models for each layer with the following configuration:

- **Input dimension:** 2048 (GPT-Neo hidden size)
- **Feature dimension:** $f = 8000$
- **L1 regularization:** $\lambda = 5.0$
- **Learning rate:** $\eta = 5 \times 10^{-5}$
- **Batch size:** 4096

4.4.2 Centroid Distance Results

Euclidean Distance Performance

Figure 4.41 shows the average centroid distances using the Euclidean metric across layers 12-16.

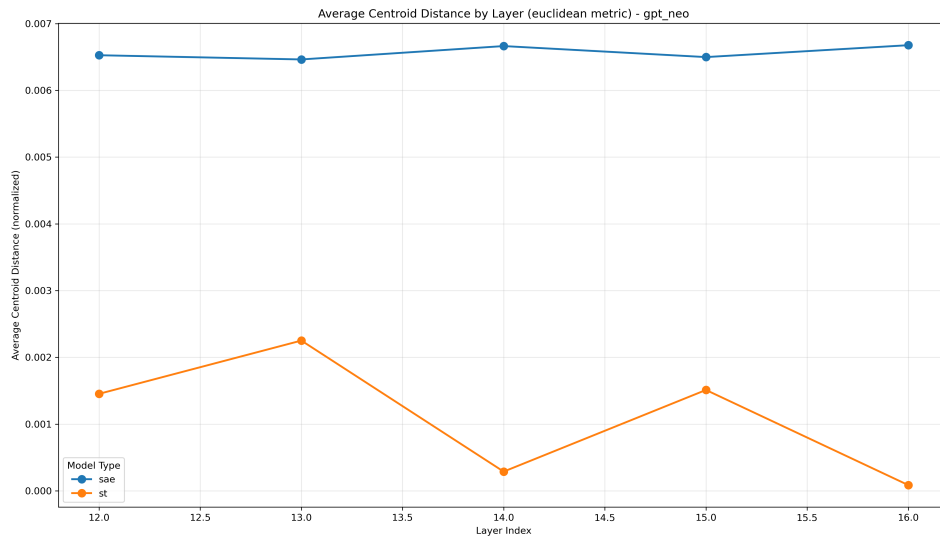


Figure 4.41: Average centroid distance by layer using Euclidean metric on GPT-Neo 1.3B representations.

Key observations:

- SAE maintains stable separation across all layers (0.0065)
- ST shows variable performance, ranging from 0.0001 to 0.0023
- SAE achieves $3\text{-}30\times$ higher Euclidean distances than ST
- The performance gap is substantially smaller than the $100\text{-}1000\times$ observed in toy datasets

Cosine Distance Performance

Figure 4.42 presents the average centroid distances using the cosine metric.

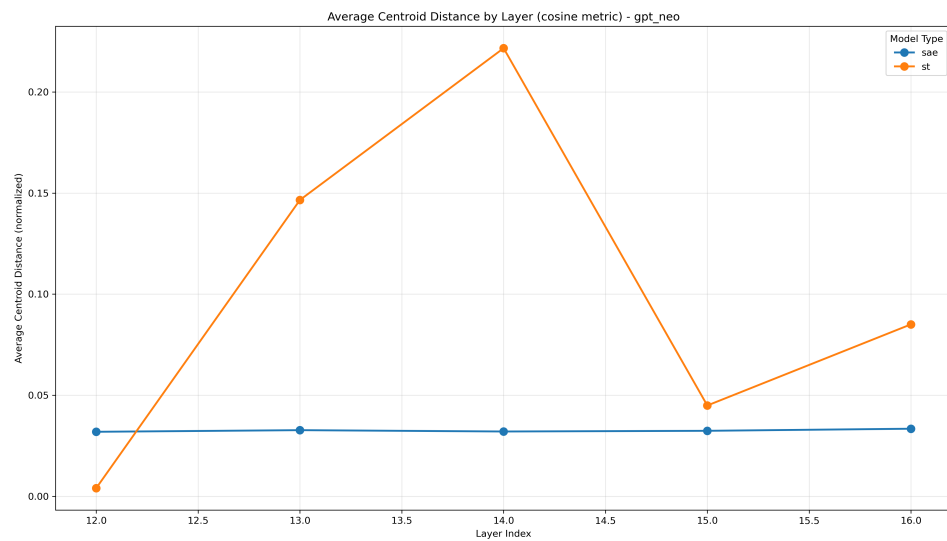


Figure 4.42: Average centroid distance by layer using cosine metric on GPT-Neo 1.3B representations.

Key observations:

- ST shows dramatic layer-dependent variation (0.003 to 0.22)
- SAE maintains consistent low separation (0.03)
- Peak ST performance at layer 14 achieves $7\times$ better separation than SAE
- The performance gap is narrower than the order-of-magnitude differences in toy datasets

4.4.3 Layer-Specific Patterns

Table 4.1 summarizes the centroid distances across all layers:

Layer	Euclidean Distance		Cosine Distance	
	SAE	ST	SAE	ST
12	0.0065	0.0015	0.031	0.003
13	0.0064	0.0023	0.030	0.147
14	0.0066	0.0003	0.030	0.220
15	0.0065	0.0015	0.032	0.045
16	0.0066	0.0001	0.033	0.082

Table 4.1: Average centroid distances by layer for both architectures on GPT-Neo 1.3B

Notable patterns:

- ST exhibits a near inverse relationship between Euclidean and cosine performance
- Layer 14 shows the most extreme behavior: minimum Euclidean but maximum cosine distance for ST
- SAE performance remains remarkably stable across all layers in both metrics

4.4.4 Comparison with Toy Dataset Results

The most striking finding is the substantial reduction in performance gaps between architectures when applied to LLM representations:

Metric	Toy Datasets	GPT-Neo
Euclidean advantage (SAE)	100-1000×	3-30×
Cosine advantage (ST)	10×	7×

Table 4.2: Performance gap comparison between toy datasets and LLM representations

This convergence indicates that the extreme separation capabilities observed in controlled settings are moderated when dealing with complex, high-dimensional representations from language models.

Part IV

Discussion and conclusion

/5

Discussion

5.1 Introduction

This thesis has presented a comprehensive theoretical and empirical comparison of Sparse Autoencoders (SAEs) and Sparse Transformers (STs) for neural network interpretability. Our investigation reveals that these architectures, while solving the same fundamental problem of learning sparse representations, exhibit complementary geometric biases that manifest consistently across diverse experimental settings. This chapter synthesizes our findings, examines their implications for mechanistic interpretability, and indicates paths for future research. The central insight emerging from our work is that architectural choices fundamentally shape not just what features are learned, but how they are geometrically organized in representation space. SAEs' unbounded ReLU activations enable extreme Euclidean separation at the cost of potential polysemanticity, while STs' normalized attention mechanisms enforce monosemantic representations through competitive dynamics but limit absolute magnitude differences. These complementary strengths suggest that the future of interpretability may lie not in choosing between architectures, but in understanding how to leverage their respective advantages.

5.2 Synthesis of Key Findings

5.2.1 Validation of Theoretical Predictions

Our empirical results validate the theoretical framework developed in Chapter 2. The predicted geometric biases manifest dramatically in controlled experiments:

1. **Euclidean Separation:** SAEs achieve $100\text{-}1000\times$ greater centroid distances than STs on toy datasets, confirming that unbounded activations enable superior absolute separation. This advantage stems directly from the architectural constraint that ST features must sum to one, fundamentally limiting their Euclidean distances to $\sqrt{2}$.
2. **Angular Discrimination:** STs consistently outperform SAEs by approximately $10\times$ in cosine distance metrics, validating our prediction that attention mechanisms naturally optimize for angular relationships. The dot-product attention computation inherently emphasizes directional similarity over magnitude.
3. **Feature Specialization Patterns:** The theoretical prediction that SAEs would exhibit variable feature sharing while STs would enforce specialization through competition is confirmed across all experimental conditions. ST features show great class-specificity even at low capacity, while SAE features transition from polysemantic to monosemantic as capacity increases.

5.2.2 Unexpected Discoveries

Beyond confirming theoretical predictions, our experiments revealed several unexpected phenomena that deepen our understanding of sparse representation learning:

Noise-Based Encoding in Sparse Transformers

The prevalence of noise-like patterns in ST features (Figures 4.27 and 4.28) suggests an unorthodox encoding strategy. Rather than learning clean geometric templates, STs appear to encode discriminative information through distributed activation patterns that manifest as apparent noise when individual features are visualized in toy datasets. This mechanism allows STs to maintain discriminative power despite the severe constraints imposed by softmax normalization.

Dataset-Dependent Regularization Effects

The dramatic difference in L1 regularization sensitivity between MNIST and Fashion-MNIST (40% vs. 15% improvement for strongest regularization) indicates that dataset complexity modulates the effectiveness of sparsity constraints. Simple datasets with clear geometric patterns benefit more from aggressive sparsity, while complex textures in Fashion-MNIST provide natural regularization that reduces the impact of explicit constraints.

Convergence in Real-World Applications

The narrowing of performance gaps when applied to GPT-Neo representations (from 100-1000 $\times$ to 3-30 $\times$ for Euclidean distances) suggests that the extreme differences observed in controlled settings may not fully translate to practical applications. This convergence has important implications for architecture selection in real-world interpretability tasks.

5.3 Implications for Mechanistic Interpretability

5.3.1 Complementary Interpretability Strategies

Our findings suggest that SAEs and STs offer fundamentally different windows into neural network computations:

- **SAEs for Concept Detection:** The ability to achieve extreme Euclidean separation makes SAEs ideal for identifying distinct concepts or features that require clear on/off behavior. Their unbounded activations can represent feature strength in absolute terms, making them suitable for understanding "how much" of a concept is present.
- **STs for Relational Understanding:** The superior angular discrimination and enforced competition make STs better suited for understanding relationships between features and their relative importance. The normalized outputs naturally represent "which feature matters most" rather than absolute magnitudes.

5.3.2 The Polysemanticity-Monosemanticity Trade-off

Our analysis reveals a fundamental trade-off in interpretability: SAEs' flexibility allows them to discover efficient shared representations (polysemanticity) that may better capture the true structure of neural network features, while STs' competitive dynamics enforce cleaner but potentially less efficient monosemantic representations.

5.3.3 Implications for LLM Interpretability

The application to GPT-Neo representations reveals important considerations for practical interpretability:

- **Layer-Specific Behaviors:** The dramatic variation in ST performance across LLM layers (cosine distances ranging from 0.003 to 0.22) suggests that different layers may benefit from different interpretability approaches. Middle layers showing peak ST performance may encode more relational information, while other layers may be better suited to SAE analysis.
- **Scalability Considerations:** The convergence of performance gaps at scale indicates that architectural choice may be less critical for large-scale applications than for understanding toy models. This suggests that hybrid approaches combining both architectures may be particularly valuable.

5.4 Limitations and Future Directions

5.4.1 Current Limitations

While our study provides comprehensive insights, several limitations should be acknowledged:

1. **Metric Limitations:** Our focus on unsupervised metrics (centroid distances) provides geometric insights but may not fully capture semantic interpretability. Future work should incorporate human evaluation of feature interpretability.
2. **Architecture Variants:** We examined canonical implementations of SAEs and STs. Variants such as Gated SAEs [23] or different attention mechanisms may exhibit different properties.

3. **Training Dynamics:** Our analysis focuses on converged models. The training dynamics and how features emerge during optimization remain unexplored.
4. **Causal Verification:** While we demonstrate that architectures learn different features, verifying that these features causally influence model behavior requires additional intervention experiments.

5.4.2 Future Research Directions

Our findings open several promising avenues for future research:

Hybrid Architectures

The complementary strengths of SAEs and STs suggest that hybrid architectures could leverage both Euclidean and angular separation. Potential approaches include:

- Parallel SAE-ST architectures with learned routing
- Sequential composition where SAE features feed into ST attention
- Adaptive architectures that switch between mechanisms based on input characteristics

Theoretical Extensions

Several theoretical questions merit further investigation:

- Formal characterization of the polysemanticity-monosemanticity transition
- Information-theoretic analysis of the noise-based encoding in STs
- Optimization landscape analysis for hybrid sparse architectures

Practical Applications

Extending our findings to practical interpretability tasks:

- Layer-specific architecture selection for LLM interpretability
- Real-time feature monitoring in deployed systems
- Integration with circuit discovery methods [10]

5.5 Broader Implications

5.5.1 For AI Safety

Our work contributes to AI safety by providing tools to better understand neural network internals. The ability to extract interpretable features is crucial for:

- Detecting potential failure modes before deployment
- Understanding model biases and their sources
- Verifying that models learn intended concepts rather than spurious correlations

5.5.2 For Machine Learning Theory

The geometric perspective on sparse representations connects interpretability to fundamental questions in representation learning:

- How do architectural biases shape learned representations?
- What is the relationship between sparsity, interpretability, and generalization?
- Can we design architectures with specific geometric properties?

/6

Conclusion

6.1 Thesis Summary

This thesis examined how architectural choices in sparse representation learning fundamentally determine the geometric organization of learned features. Through comprehensive analysis of Sparse Autoencoders and Sparse Transformers, we demonstrated that the mechanism of sparsity, whether through ReLU thresholding or softmax competition, shapes not only what features are learned but how they relate to each other in representation space.

6.2 Core Contributions

Our central contribution is the establishment of a geometric framework for understanding sparse neural network interpretability. We proved both theoretically and empirically that:

1. Architectural constraints create inescapable geometric biases: SAEs operate in unbounded positive orthants while STs are confined to the probability simplex
2. These geometric differences manifest as complementary capabilities: extreme Euclidean separation versus superior angular discrimination

6.3 Resolving the Central Question

This thesis answered the fundamental question: *Are different sparse representation architectures merely implementation details, or do they learn fundamentally different features?* Our work definitively establishes that architectural choices are not neutral, they profoundly shape the learned representations through their geometric biases. The answer has immediate practical implications: there might not be a universally superior architecture for neural network interpretability. Instead, the choice depends on whether the interpretability goal requires understanding absolute magnitudes (favoring SAEs) or relative relationships (favoring STs).

6.4 Broader Significance

Beyond the specific architectures studied, this work establishes geometric analysis as a powerful lens for understanding neural network interpretability methods. The principles derived, that optimization constraints shape representation geometry, that sparsity mechanisms determine feature relationships, and that theoretical advantages may be moderated at scale, extend beyond SAEs and STs to future interpretability approaches.

6.5 Final Reflection

As we develop increasingly powerful AI systems, the ability to peer inside their learned representations becomes ever more critical. This thesis demonstrates that our choice of interpretability tools fundamentally shapes what we can see and understand about these systems. By recognizing that different architectures provide complementary views rather than competing alternatives, we move closer to comprehensive understanding of neural network internals. The geometric perspective developed here offers a principled foundation for future interpretability research. Just as microscopes and telescopes provide different but complementary views of the physical world, Sparse Autoencoders and Sparse Transformers offer distinct but equally valuable windows into the computational world of neural networks. Understanding their respective strengths and limitations is essential for the responsible development of interpretable AI systems.

Bibliography

- [1] Dario Amodei et al. “Concrete Problems in AI Safety.” In: *arXiv preprint arXiv:1606.06565* (2016).
- [2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. “Neural Machine Translation by Jointly Learning to Align and Translate.” In: *arXiv preprint arXiv:1409.0473* (2014).
- [3] Leo Breiman. “Random forests.” In: *Machine learning* 45 (2001), pp. 5–32.
- [4] Trenton Bricken et al. “Towards Monosemanticity: Decomposing Language Models With Dictionary Learning.” In: *Transformer Circuits Thread* (2023).
- [5] Tom Brown et al. “Language Models are Few-Shot Learners.” In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 1877–1901.
- [6] Emmanuel J. Candès, Justin Romberg, and Terence Tao. “Robust Uncertainty Principles: Exact Signal Reconstruction from Highly Incomplete Frequency Information.” In: *IEEE Transactions on Information Theory* 52.2 (2006), pp. 489–509.
- [7] Hoagy Cunningham et al. “Sparse Autoencoders Find Highly Interpretable Features in Language Models.” In: *arXiv preprint arXiv:2309.08600* (2023).
- [8] David L. Donoho and Michael Elad. “Optimally Sparse Representation in General (Nonorthogonal) Dictionaries via ℓ^1 Minimization.” In: *Proceedings of the National Academy of Sciences* 100.5 (2003), pp. 2197–2202.
- [9] Finale Doshi-Velez and Been Kim. “Towards A Rigorous Science of Interpretable Machine Learning.” In: *arXiv preprint arXiv:1702.08608* (2017).
- [10] Nelson Elhage et al. “A Mathematical Framework for Transformer Circuits.” In: *Transformer Circuits Thread* (2021).
- [11] Nelson Elhage et al. “Toy Models of Superposition.” In: *arXiv preprint arXiv:2209.10652* (2022).
- [12] Wes Gurnee et al. “Finding Neurons in a Haystack: Case Studies with Sparse Probing.” In: *arXiv preprint arXiv:2305.01610* (2023).
- [13] Robert Huben, Hoagy Cunningham, Timnit Eloundou, et al. “Sparse Autoencoders Enable Scalable and Reliable Circuit Identification in Language Models.” In: *arXiv preprint* (2024).

- [14] Evan Hubinger et al. “Sleeper Agents: Training Deceptive LLMs that Persist Through Safety Training.” In: *arXiv preprint arXiv:2401.05566* (2024).
- [15] Been Kim et al. “Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV).” In: *International Conference on Machine Learning*. 2018, pp. 2668–2677.
- [16] Diederik P Kingma and Jimmy Ba. “Adam: A method for stochastic optimization.” In: *arXiv preprint arXiv:1412.6980* (2014).
- [17] Scott M. Lundberg and Su-In Lee. “A Unified Approach to Interpreting Model Predictions.” In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017, pp. 4765–4774.
- [18] Tom McCoy, Ellie Pavlick, and Tal Linzen. “Right for the Wrong Reasons: Diagnosing Syntactic Heuristics in Natural Language Inference.” In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. 2019, pp. 3428–3448.
- [19] Guido F. Montúfar et al. “On the Number of Linear Regions of Deep Neural Networks.” In: *Advances in Neural Information Processing Systems* 27 (2014), pp. 2924–2932.
- [20] Neel Nanda et al. “Progress measures for grokking via mechanistic interpretability.” In: *arXiv preprint arXiv:2301.05217* (2023).
- [21] Chris Olah et al. “Zoom In: An Introduction to Circuits.” In: *Distill* 5.3 (2020), e00024.001.
- [22] Bruno A. Olshausen and David J. Field. “Emergence of Simple-cell Receptive Field Properties by Learning a Sparse Code for Natural Images.” In: *Nature* 381.6583 (1996), pp. 607–609.
- [23] Senthooran Rajamanoharan et al. “Improving Dictionary Learning with Gated Sparse Autoencoders.” In: *arXiv preprint arXiv:2404.16014* (2024).
- [24] Tilman R  uker et al. “Toward Transparent AI: A Survey on Interpreting the Inner Workings of Deep Neural Networks.” In: *arXiv preprint arXiv:2207.13243* (2023).
- [25] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. ““Why Should I Trust You?”: Explaining the Predictions of Any Classifier.” In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2016, pp. 1135–1144.
- [26] Lee Sharkey, Dan Braun, and Beren Millidge. “Taking Features Out of Superposition with Sparse Autoencoders.” In: *AI Alignment Forum* (2022).
- [27] Paul Smolensky. “On the Proper Treatment of Connectionism.” In: *Behavioral and Brain Sciences*. Vol. 11. 1. 1988, pp. 1–23.
- [28] Adly Templeton et al. “Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet.” In: *Anthropic* (2024).
- [29] Ashish Vaswani et al. “Attention Is All You Need.” In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017, pp. 5998–6008.

- [30] Matthew D. Zeiler and Rob Fergus. “Visualizing and Understanding Convolutional Networks.” In: *European Conference on Computer Vision*. Springer. 2014, pp. 818–833.

